

National Taipei University of Technology
Object-Oriented Analysis and Design
Homework #7

arch-checker

A Fluent Architectural Style Compliance Tool for Java

Team Member:

113598009 資工碩二 李俊威

Table of Contents

1. Requirement Document	3
1.1 Change History	3
1.2 Problem Statement	4
1.3 Summary of System Features	4
1.4 Use Case Diagram.....	4
1.5 Use Cases	5
1.6 Non-Functional Requirements and Constraints	10
1.7 Glossary	10
1.8 Software Environments.....	11
2. Domain Model	11
2.1 Domain Class Diagram (Concepts)	11
2.2 Associations	13
2.3 Attributes.....	14
3. Design	15
3.1 Logical Architecture	15
3.2 Use-Case Realizations with GRASP Patterns.....	16
3.3 Design Class Model	25
4. Implementation Class Model	26
4.1 Implementation Class Diagram.....	26
4.2 Differences between Design and Implementation Class Model	27
4.3 Calculate Line of Code	27
5. Programming.....	28
5.1 Snapshots of System Execution	28
5.2 Source Code Listing.....	30
6. Unit Testing	36
6.1 Snapshots of Testing Result.....	36
6.2 Unit Testing Code Listing.....	36
7. Revision Based on TA Comments, Midterm Review, and Findings	42
8. Measurement.....	43

1. Requirement Document

1.1 Change History

Version	Date	Author	Description
1.0	2026-03-03	李俊威	Initial submission (HW1) — Problem Statement
2.0	2026-03-12		Add System Features, Use Cases, NFR, Glossary (HW2)
3.0	2026-04-01		Add Domain Model; revise HW2: condense Problem Statement, rename SF-XX to FEA-XX, remove CLI commands from Use Cases, generalize error messages, remove implementation details
4.0	2026-04-17		Add Design (Logical Architecture, Use-Case Realizations with GRASP, Design Class Model); revise HW3: remove non-system-state items from UC-02/UC-03 preconditions, reformat Domain Model descriptions as tables
5.0	2026-04-22		Revise HW4: compact UC-01/UC-04 Sequence Diagrams, redraw Design Class Diagram, add Class Specifications table
6.0	2026-04-29		新增 Implementation Class Model、Programming、Unit Testing 三章節（HW5）；系統以 Maven 專案交付，使用 JavaParser、SnakeYAML、picocli、JUnit 5。
7.0	2026-05-05		重新命名介面與類別以更精確反映職責：ProfileRepository→ProfileLoader、SuppressionRepository→SuppressionStore、ArchitectureConstraint→ComplianceRule（套件 domain.constraint→domain.rule）；同步更新 Implementation Class Diagram、Sequence Diagrams 與內文敘述。
8.0	2026-05-26		Iteration II：選定 UC-02 Define Style Profile 與 UC-03 Check with Interactive Fix Suggestions 為本迭代要實作之 use cases；於 Domain Model 新增 Violation.suggestion [0..1] 屬性以支援 UC-03 顯示修正建議；依 HW6 規定移除 Design／Implementation／Programming／Unit Testing 四章節，留待 HW7 提交。
9.0	2026-06-17		實作 Iteration II：新增 UC-02 定義風格設定檔、UC-03 互動式違規修正；6 個新類別，28 個 JUnit 5 測試全部通過。

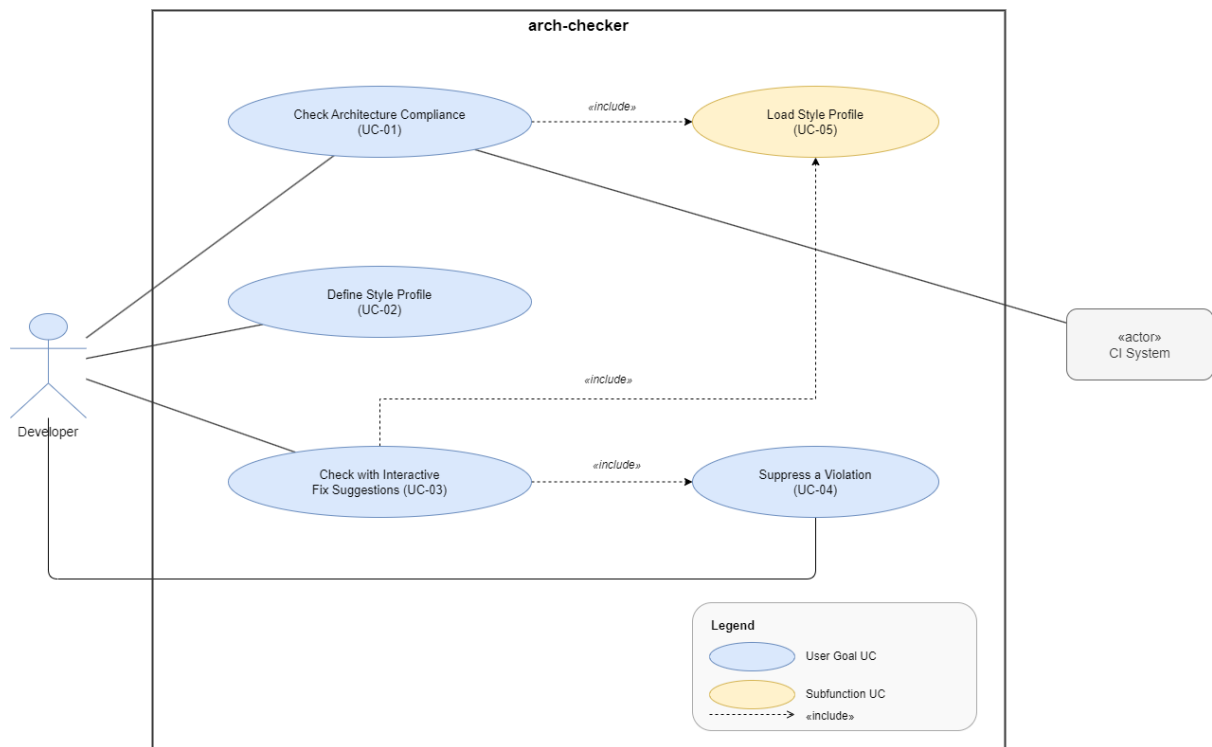
1.2 Problem Statement

隨著軟體系統規模擴大與 AI 輔助程式碼生成的普及，人工驗證架構風格合規性日益困難，成為開發週期的瓶頸。現有工具（如 ArchUnit）雖可將架構限制寫成測試規則，但使用門檻較高，開發者需熟悉底層 API 並撰寫大量樣板程式碼，缺乏以風格為中心的高階抽象。arch-checker 是一套專為 Java 開發者設計的命令列工具，用於自動驗證 Java 專案的架構風格合規性。使用者可透過宣告式 API 以 YAML 定義架構風格設定檔（Style Profile），將命名、依賴、繼承等規則封裝為可重複使用的風格單元。系統透過 Java 原始碼的 AST 解析，精確偵測結構性違規，並產出含檔案路徑、行號與錯誤描述的完整報告，適用於本地開發、程式碼審查與持續整合情境。

1.3 Summary of System Features

ID	Feature Name	Priority	Description
FEA-01	Architecture Compliance Checking	High	掃描 Java 專案目錄，依使用者定義的 Style Profile 偵測違規
FEA-02	Style Profile Definition via YAML	High	使用者以 YAML 格式定義任意架構風格規則（DependencyRule、NamingRule、SupertypeRule、PackageRule）；工具不預設任何內建風格
FEA-03	Violation Report Generation	High	輸出含檔案路徑、行號、規則 ID、說明的違規報告
FEA-04	Multiple Report Formats	Low	支援 Console 與 JSON 兩種輸出格式
FEA-05	Violation Suppression	Medium	標記特定違規為「已知且接受」，下次不再回報

1.4 Use Case Diagram



Actors:

Developer (Primary)：使用工具定義架構規則並檢查 Java 專案的開發者

CI System (Secondary)：在 CI/CD pipeline 中自動觸發合規檢查，依賴 exit code 判斷結果

UC	Name	Level	Format	Triggered by
UC-01	Check Architecture Compliance	User Goal	Fully Dressed	Developer / CI System
UC-02	Define Style Profile	User Goal	Fully Dressed	Developer
UC-03	Check with Interactive Fix Suggestions	User Goal	Fully Dressed	Developer
UC-04	Suppress a Violation	User Goal	Brief	Developer
UC-05	Load Style Profile	Subfunction	Brief	UC-01 / UC-03 (include)

1.5 Use Cases

UC-01: Check Architecture Compliance (Fully Dressed)

Use Case	UC-01
Use Case Name	Check Architecture Compliance
Scope	arch-checker CLI

Level	User Goal
Primary Actor	Developer
Stakeholders and Interests	Developer：想要取得準確且詳細的違規報告，包含檔案路徑與行號，以便快速找出不符合規範的類別，避免費時的人工審查。 Team Lead / Architect：想確保整個程式碼庫符合團隊定義的架構規則，零違規代表架構一致性獲得維護。 CI System（次要）：想要可靠的 POSIX exit code（0/1/2）來判斷 pipeline 是否通過，不需要解讀報告內文。
Preconditions	1. arch-checker 已安裝並可在系統中存取 2. 目標 Java 專案目錄存在且可讀取 3. Style Profile YAML 檔案存在且語法正確
Success Guarantee	系統產生完整的 ViolationReport，包含：已檢查檔案數、違規清單、suppressedCount；並回傳 POSIX exit code（0 = pass，1 = violations found）
Main Success Scenario	1. Developer 發出對指定專案目錄進行架構合規檢查的要求，並指定要使用的 Style Profile 2. 系統確認專案目錄存在 3. 系統讀取並解析指定的 Style Profile YAML 檔案 4. 系統遞迴掃描專案目錄下所有 .java 檔案 5. 系統以 JavaParser 將每個 .java 檔解析為 AST（CompilationUnit） 6. 系統對每個 CompilationUnit 套用 Profile 中所有 ComplianceRule 規則 7. 系統收集所有 Violation，每筆包含：檔案路徑、行號、規則 ID、說明文字 8. 系統過濾已被 suppress 的 Violation 9. 系統以 Console 格式輸出 ViolationReport（含違規清單與 summary） 10. 系統回傳 exit code：無違規則 0，有違規則 1
Extensions	2a. 專案目錄不存在：系統輸出錯誤訊息，exit code 2，終止 3a. Style Profile 檔案不存在：系統輸出錯誤訊息，exit code 2，終止 3b. YAML 語法錯誤：系統輸出 YAML 解析錯誤與行號，exit code 2，終止 3c. YAML 語法正確但規則定義無效（如未知的 rule type）：系統輸出 validation 錯誤清單，exit code 2，終止 4a. 專案目錄下無 .java 檔：系統輸出警告訊息，exit code 0，終止

Special Requirements	5a. 某 .java 檔語法錯誤無法解析：系統記錄警告訊息，跳過該檔案，繼續處理其餘檔案 8a. 所有 Violation 皆已被 suppress：報告顯示全部通過（含已抑制數量），exit code 0
	NFR-01（Performance）：掃描 1,000 個 .java 檔案必須在 10 秒內完成（一般開發機） NFR-02（Usability）：所有錯誤訊息必須包含：錯誤原因、位置（檔案/行號）、建議修正動作 NFR-05（CI Compatibility）：exit code 遵循 POSIX 標準，可直接整合 GitHub Actions / Jenkins
	*a. Style Profile 接受任意 YAML 檔案路徑，不支援內建名稱 *b. 輸出格式可切換為 JSON，預設為 Console 文字格式 4a. Java 檔案掃描採遞迴深度優先走訪
Technology and Data Variations List	
Frequency of Occurrence	CI 環境：每次 push 觸發，每日可達數十次；本地開發：每小時數次
Miscellaneous	是否需支援 multi-module Maven 專案（多個 src/main/java 子目錄）？ 大型專案（>10,000 個 .java 檔）是否需要增量掃描機制？

UC-02: Define Style Profile (Brief) ← Iteration II（本迭代實作）

Developer 執行 arch-checker profile init <outputPath>，系統於指定路徑產生含預設規則範例與欄位說明的 YAML 模板，供 Developer 手動編輯後用於 UC-01 或 UC-03 合規檢查。輸出目錄不存在時系統自動建立；若路徑已有檔案則覆蓋。

UC-03: Check Architecture Compliance with Interactive Fix Suggestions (Fully Dressed) ← Iteration II（本迭代實作）

Use Case	UC-03
Use Case Name	Check Architecture Compliance with Interactive Fix Suggestions
Scope	arch-checker CLI
Level	User Goal
Primary Actor	Developer
Stakeholders and Interests	Developer：想逐條審視每個 Violation 並當場決定是否 suppress，希望系統提供可操作的修復建議，而不只是列出錯誤清單。 Team Lead：想知道哪些 Violation 是「已知且接受」的技術債，並有持久化且可稽核的 suppression 紀錄，而非無聲地被

	略過。
Preconditions	1. 同 UC-01 的前置條件 2. Terminal 支援互動式輸入（非 pipe 重定向的 non-TTY 模式）
Success Guarantee	對每條 Violation，三種結果之一成立：(a) Suppression 記錄寫入 .arch-checker-suppress.yaml，(b) Violation 保留在報告中，(c) 流程中止；最終 ComplianceReport 反映所有決策結果
Main Success Scenario	1. Developer 發出以互動模式對指定專案進行架構合規檢查的要求，並指定 Style Profile 2. 系統完成掃描流程（同 UC-01 Step 2–7）：走訪檔案、解析 AST、套用規則、收集所有 Violation 3. 系統展示第一條 Violation，顯示：檔案路徑與行號、違反的 ComplianceRule ID 與說明、RefactoringSuggestion（若存在） 4. 系統提示 Developer 選擇動作（Suppress? [y/n/q]） 5. Developer 輸入 y：系統將此 Violation（rule ID、檔案路徑、行號、時間戳記）記錄至 .arch-checker-suppress.yaml 6. 系統展示下一條 Violation（回到 Step 3） 7. 所有 Violation 處理完畢後，系統輸出摘要：已檢查的檔案數、剩餘 Violation 數、已接受的 Suppression 數 8. 系統回傳 exit code（0 = 無未接受 Violation，1 = 有未接受 Violation）
Extensions	3a. 此 Violation 有 RefactoringSuggestion：系統在顯示 [y/n/q] 提示前，額外顯示建議操作 3b. 掃描後無 Violation：系統直接輸出全部通過訊息，不進入互動迴圈，exit code 0 5a. Developer 輸入 n：此 Violation 保留在最終報告中，系統繼續下一條（Step 6） 5b. Developer 輸入 q：系統中止互動迴圈，輸出已處理部分的 partial report，exit code 1 5c. Developer 輸入非法值（非 y/n/q）：系統輸出 Invalid input. Please enter y, n, or q:，重新等待輸入 8a. 所有 Violation 均被 suppress（全部輸入 y）：報告顯示全部通過（含已抑制數量），exit code 0
Special Requirements	NFR-02（Usability）：互動提示須清晰易懂，Developer 無需查閱文件即能理解 [y/n/q] 各選項的含義 NFR-06（Testability）：互動流程的合規性檢查邏輯必須可透過 unit test 獨立驗證，不依賴完整的 CLI 環境
Technology and	*a. Suppression 記錄寫入 .arch-checker-suppress.yaml，包含：

Data Variations List	rule ID、reason（預設為 suppressed interactively）、timestamp 4a. [y/n/q] 輸入大小寫不敏感（Y、Yes 均視為接受） *b. 若 stdout 被 pipe 重定向（non-TTY），系統自動降級為非互動批次模式並輸出警告
Frequency of Occurrence	本地開發初次建立 suppression 清單時使用，通常每個新專案執行一次；後續違規出現時按需使用，整體低頻
Miscellaneous	是否需要提供 --accept-all 選項，一次 suppress 所有 Violation？ Suppression 是否應設有效期限，避免長期堆積技術債？

UC-04: Suppress a Violation (Fully Dressed)

Use Case	UC-04
Use Case Name	Suppress a Violation
Scope	arch-checker CLI
Level	Subfunction
Primary Actor	（由 UC-03 互動迴圈驅動，Developer 以選擇 y 間接觸發）
Stakeholders and Interests	Developer：希望在不修改程式碼的情況下，將已知且接受的違規排除於後續合規報告之外。
Preconditions	目標 Violation 已存在於本次 ViolationReport 中 .arch-checker-suppress.yaml 路徑可存取（檔案不存在時由系統建立）
Success Guarantee	Violation 的 rule ID、檔案路徑、行號與時間戳記已持久化至 .arch-checker-suppress.yaml；UC-01／UC-03 下次執行時該違規不再出現於報告
Main Success Scenario	1. UC-03 互動迴圈收到 Developer 選擇 y，FixCommand 委派 SuppressionService.suppress(violation, profile, suppressFile) 執行 2. 系統從 .arch-checker-suppress.yaml 載入現有 Suppression 清單 3. 系統建立新的 Suppression 記錄（rule ID、檔案路徑、行號、"Suppressed interactively"、當前 ISO-8601 時間戳記） 4. 系統將更新後的 Suppression 清單寫回 .arch-checker-suppress.yaml
Extensions	*a. .arch-checker-suppress.yaml 不存在：系統建立新檔後寫入，正常結束 *b. 磁碟寫入失敗（空間不足或無寫入權限）：系統拋出例外，錯誤向上傳遞至 UC-03 並輸出錯誤訊息

Special Requirements	NFR-03 (Traceability)：每筆 Suppression 記錄須含 rule ID、file path、line number 及 ISO-8601 timestamp，以便日後稽核
Technology and Data Variations	*a. Suppression 清單存於 YAML 格式（以 SnakeYAML 讀寫），預設路徑為 .arch-checker-suppress.yaml
Frequency of Occurrence	每次 UC-03 互動迴圈中 Developer 選擇 y 時觸發；頻率取決於本次 ViolationReport 的違規數量與 Developer 的決策
Miscellaneous	suppress 指令亦可由 Developer 直接執行以獨立觸發本 UC

UC-05: Load Style Profile (Brief — Subfunction)

UC-01 與 UC-03 的 <<include>> 子流程。系統讀取使用者指定的 Style Profile YAML 檔，解析並驗證規則定義。成功後將 Profile 載入記憶體，供各 rule checker 使用。

1.6 Non-Functional Requirements and Constraints

ID	Category	Requirement
NFR-01	Performance	完整的 compliance check (UC-01) 掃描 1,000 個 .java 檔案必須在 10 秒內完成（一般開發機）
NFR-02	Usability	所有錯誤訊息必須包含：錯誤原因、位置（檔案路徑 / 行號）、建議修正動作
NFR-03	Portability	工具需在 JDK 17+ 環境（Windows / macOS / Linux）上運行，不依賴特定 IDE
NFR-04	Extensibility	系統應支援在不修改既有合規檢查流程的前提下，擴充對新架構規則類型的支援
NFR-05	CI Compatibility	exit code 遵循 POSIX 標準（0 = success，非 0 = failure），可直接整合 GitHub Actions 與 Jenkins
NFR-06	Testability	每個架構規則的合規性檢查邏輯必須可透過 unit test 獨立驗證，不依賴完整的 CLI 環境

1.7 Glossary

Term	Definition
Style Profile	使用者以 YAML 格式定義的架構規則集合，描述特定專案的架構約束；工具本身不提供任何內建 Profile
ComplianceRule	Style Profile 中的單條約束規則，為 NamingRule、DependencyRule、SupertypeRule、PackageRule 的抽象父類別
SupertypeRule	一種 ComplianceRule，要求特定 package 內的 class 必須

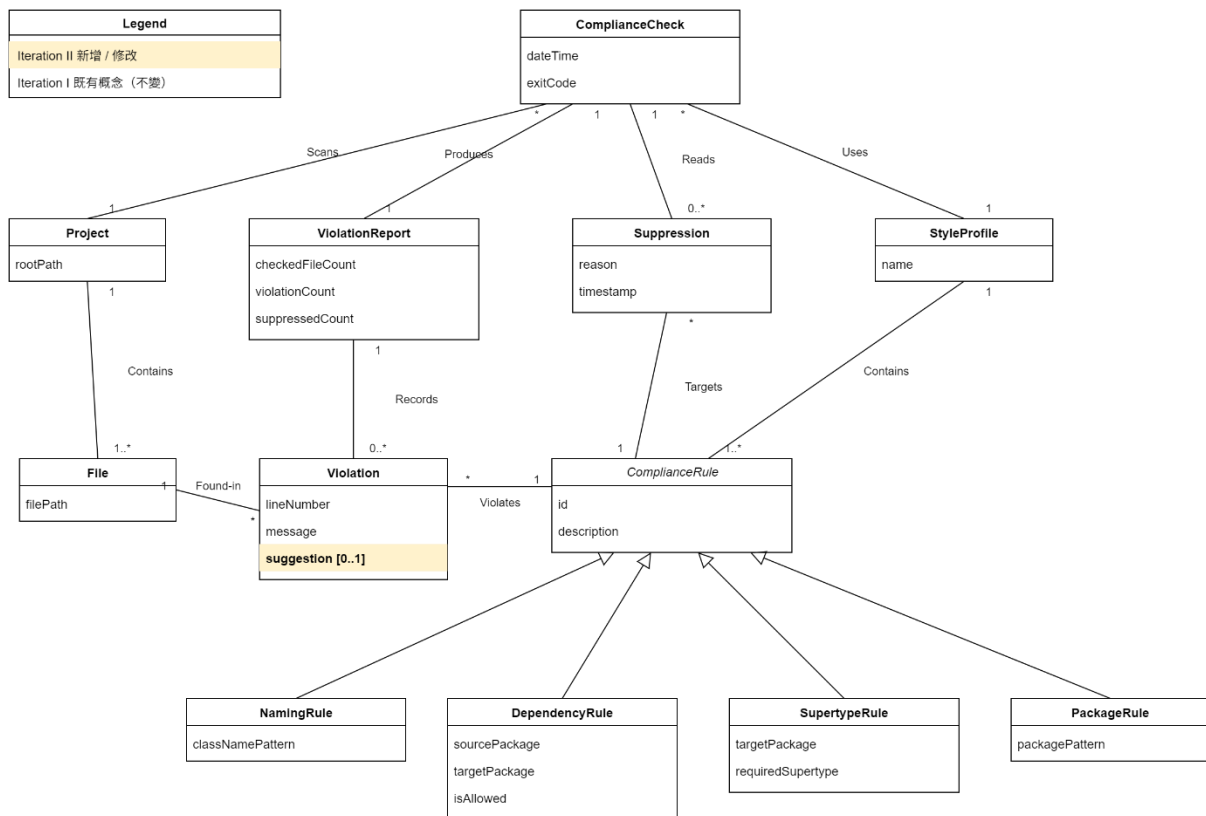
Term	Definition
	implement 指定 interface 或 extend 指定 class；以 simple name 比對，不進行 fully qualified name resolution
Violation	一筆架構違規記錄，包含：檔案路徑、行號、規則 ID、說明文字
ViolationReport	單次 check 執行的完整輸出，彙整所有 Violation 與 summary
AST (Abstract Syntax Tree)	Java 原始碼的抽象語法樹表示，由 JavaParser 解析產生，供各 ConstraintChecker 進行分析
CompilationUnit	JavaParser 解析單一 .java 檔案後產生的根節點物件
Suppression	開發者明確標記「此 Violation 已知且接受」的機制，使其在後續報告中不再出現
exit code	CLI 程式結束時回傳給 shell 的整數碼（0 = pass，1 = violations found，2 = error）
Package Pattern	規則定義中用來比對 Java package 名稱的萬用字元字串（如 *.ui.*）；只允許英數字、*、.
suggestion	針對單筆 Violation 的修正建議文字，於互動模式（UC-03）顯示給開發者；為 Violation 的可選屬性（[0..1]），非獨立概念類別。對應 NFR-02 之「建議修正動作」。

1.8 Software Environments

Item	Choice	Rationale
Development Language	Java 17 (JDK 17+)	LTS 版本穩定
AST Parser	JavaParser 3.x	純 Java，API 直覺，無需 JDK tools
CLI Framework	picocli	輕量，annotation-driven，與 Java 17 相容
Profile Format	YAML (SnakeYAML)	可讀性高，易於編輯
Test Framework	JUnit 5	unit testing
Build Tool	Maven	依賴管理方便

2. Domain Model

2.1 Domain Class Diagram (Concepts)



上圖為 arch-checker 的 Domain Class Diagram。識別出的概念類別如下：

已採納的概念類別與理由：

Concept	Rationale for Inclusion
ComplianceCheck	代表一次完整的架構合規檢查作業，對應 UC-01/UC-03 中 Developer 對專案發出檢查要求的動作。系統需記錄每次檢查的時間與結果。
StyleProfile	使用者以 YAML 定義的架構規則集合，包含多條 ComplianceRule。系統在 UC-01 載入 Profile 後，於 UC-02 逐條驗證其中的約束規則。
ComplianceRule	StyleProfile 中的單條約束規則（抽象概念），為 Profile 的組成元素。設為抽象類別，因其具有四個具體子類型，各自對應不同的架構約束面向。
NamingRule / DependencyRule / SupertypeRule / PackageRule	ComplianceRule 的四種具體子類型，分別檢查類別命名慣例、套件間依賴方向、繼承關係要求與套件結構規範。UC-02 驗證流程中依規則類型分派不同的檢查邏輯。
Project	被檢查的目標 Java 專案，UC-01 中 Developer 指定的掃描對象。系統需知道專案路徑以定位其中的 File。
File	專案中的 .java 原始碼檔案，為 Project 的組成元素。每個 Violation 需關聯到特定的 File 以定位違規位置。
ViolationReport	單次檢查的完整報告，為 ComplianceCheck 的產出物。彙整所

Concept	Rationale for Inclusion
	有 Violation 並提供統計摘要，供 Developer 或 CI 系統判讀結果。
Violation	單筆架構違規記錄，記載哪個 File 違反了哪條 ComplianceRule。為 ViolationReport 的明細項目。
Suppression	開發者標記「已知且接受」的違規抑制記錄，對應 UC-04 的功能。讓團隊可以在不修改程式碼的情況下，將特定違規排除於報告之外。

排除的候選概念與理由：

Excluded Candidate	Reason for Exclusion
CompilationUnit / AST	這是 JavaParser 函式庫的軟體產物（software artifact），屬於實作層面的概念，不應出現在 Domain Model 中。
ProfileLoader / ProfileValidator	這些是軟體類別（software classes），負責解析與驗證 Profile 的內部行為，非領域中的真實概念。
ConsoleReport / JSONReport	Console 與 JSON 是 ViolationReport 的輸出格式（SF-04），屬於呈現方式而非領域概念，不需獨立建模。
Developer / CISystem	這些是外部 Actor，不屬於系統內部的領域概念。
ExitCode	exit code 是數值型態，適合作為 ComplianceCheck 的屬性而非獨立的概念類別。
ProfileTemplate	UC-02 產生的 YAML 模板是 StyleProfile 的輸出／呈現形式（與被排除的 ConsoleReport／JSONReport 同類），屬實作層產物，非領域概念。
ValidationResult / ValidationError	UC-02 驗證流程的產出。iteration I 已將負責解析與驗證 Profile 的 ProfileValidator 列為軟體類別，其產出同屬實作層，不需建模為領域概念。
Suggestion（升級為類別）	UC-03 的修正建議在本領域僅為一段文字，無自身結構、身份與關聯。依「純資料型別值即屬性」原則，應為 Violation 的屬性 suggestion [0..1]；升級為獨立類別屬過度建模（over-modeling）。
InteractiveSession / FixDecision	UC-03 的 [y/n/q] 逐條互動為控制流程，非「需要記憶的事物」；其結果（寫入 Suppression 或保留 Violation）已由既有概念建模，無須另立會話類別。

2.2 Associations

關聯的建立依據「需要記憶的關係」原則（need-to-remember associations）。下表

列出各關聯的名稱、對應數量與建立理由：

Association	Cardinality	Rationale
ComplianceCheck Uses StyleProfile	* — 1	每次檢查使用一份 Profile；同一份 Profile 可被多次檢查使用。
ComplianceCheck Scans Project	* — 1	每次檢查掃描一個專案；同一專案可被多次檢查。
ComplianceCheck Produces ViolationReport	1 — 1	每次檢查產出一份報告，一對一關係。
ComplianceCheck Reads Suppression	1 — 0..*	檢查時讀取已存在的 Suppression 記錄來過濾結果。
StyleProfile Contains ComplianceRule	1 — 1..*	一份 Profile 至少包含一條規則，為組合關係。
Project Contains File	1 — 1..*	專案包含一至多個 Java 檔案，為組合關係。
ViolationReport Records Violation	1 — 0..*	報告記錄零至多筆違規，為報告的明細項目。
Violation Violates ComplianceRule	* — 1	每筆違規對應一條被違反的規則。
Violation Found-in File	* — 1	每筆違規發生在特定的 Java 檔案中。
Suppression Targets ComplianceRule	* — 1	每筆抑制記錄對應一條特定規則。

此外，NamingRule、DependencyRule、SupertypeRule、PackageRule 為 ComplianceRule 的泛化（generalization）子類型，以空心三角形箭頭表示繼承關係。

2.3 Attributes

屬性的選擇遵循「在 Domain Model 中聚焦於資料型別屬性」的原則，避免將複雜概念當作屬性、避免以外鍵（foreign key）代替關聯。各概念類別之屬性整理如下：

Class	Attributes
ComplianceCheck	dateTime（檢查執行時間）、exitCode（結果碼：0=pass, 1=violations, 2=error）
StyleProfile	name（Profile 名稱，用於識別）
ComplianceRule	id（規則識別碼）、description（規則說明文字）
NamingRule	classNamePattern（類別命名的正規表達式 pattern）
DependencyRule	sourcePackage（來源套件 pattern）、targetPackage（目標套件 pattern）、isAllowed（是否允許此依賴方向）

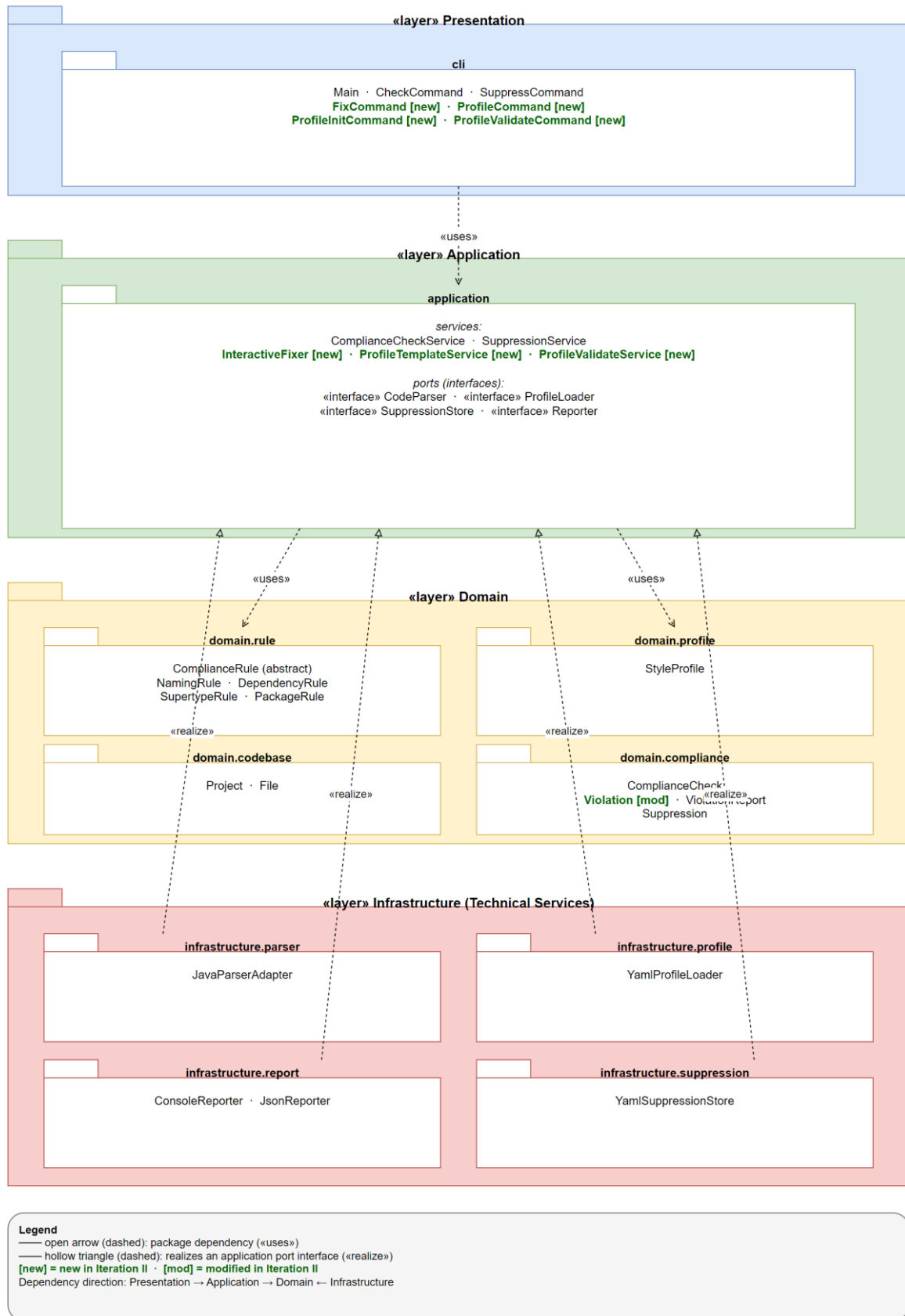
Class	Attributes
SupertypeRule	targetPackage（適用的套件 pattern）、requiredSupertype（要求繼承或實作的型別名稱）
PackageRule	packagePattern（套件結構的萬用字元 pattern）
Project	rootPath（專案根目錄路徑）
File	filePath（檔案路徑）
ViolationReport	checkedFileCount（已檢查檔案數）、violationCount（違規總數）、suppressedCount（已抑制數量）
Violation	lineNumber（發生行號）、message（違規說明）、 suggestion（針對該筆違規的修正建議，可選 [0..1]）
Suppression	filePath（被抑制違規所在檔案路徑）、lineNumber（被抑制違規所在行號）、reason（抑制原因）、timestamp（抑制時間戳記）

3. Design

3.1 Logical Architecture

arch-checker 為單一使用者的 CLI 工具（無 GUI、無資料庫後端）。本節以 UML package diagram 方式呈現 Iteration II 系統的組織架構，並說明各 package 之職責與相依方向。系統分為四個水平層：CLI 層使用 picocli 框架處理指令；Application 層定義業務邏輯與介面（CodeParser、ProfileLoader、SuppressionStore、Reporter）；Domain 層包含純粹的領域物件，無對外依賴；Infrastructure 層實作 Application 層所定義的 port 介面，遵循相依反轉原則（Dependency Inversion Principle）。Iteration II 於 CLI 層新增 FixCommand、ProfileCommand、ProfileInitCommand、ProfileValidateCommand，其中 FixCommand 負責 UC-03 互動邏輯（顯示違規、讀取決策），並委派 SuppressionService 執行 suppress 流程；於 Application 層新增 ProfileTemplateService、ProfileValidateService、SuppressionService。

arch-checker Logical Architecture (Package Diagram — Iteration II)



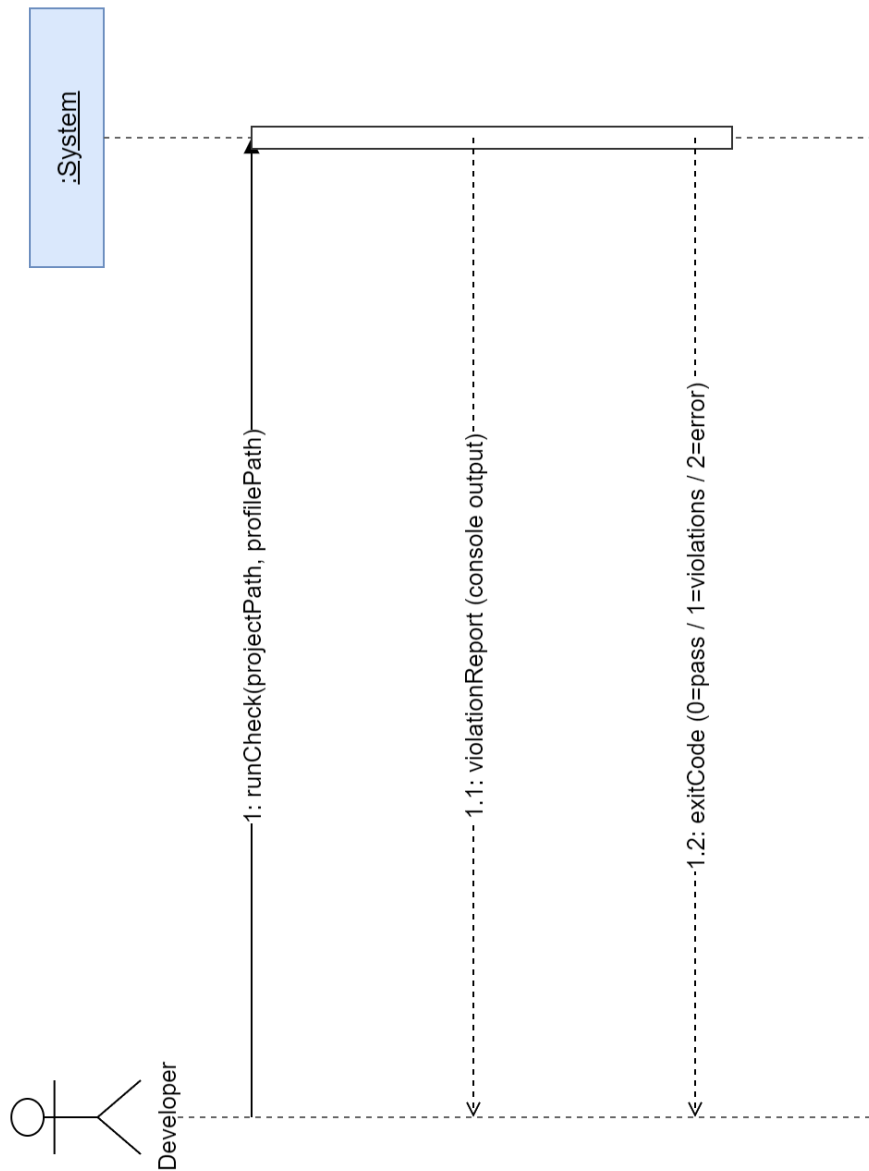
3-1 Logical Architecture (UML Package Diagram)

3.2 Use-Case Realizations with GRASP Patterns

3.2.1 System Sequence Diagram — UC-01 Check Architecture Compliance

Developer 對 :System 發出單一系統事件 `runCheck(projectPath, profilePath)`，系統執行所有啟用的合規規則並回傳 `ViolationReport`，最終以 `exitCode` 結束程序。

System Sequence Diagram — UC-01 Check Architecture Compliance



事件說明：Developer 於終端機呼叫 `arch-checker` 進行架構合規檢查。
System 載入 Profile、解析 Java 原始碼、執行規則驗證並產生 `ViolationReport`，
最後以 console 輸出報告並回傳 `exit code` 供 CI 判讀。

圖 3-2.1 UC-01 Check Architecture Compliance — System Sequence Diagram

3.2.2 Sequence Diagram with GRASP Patterns — UC-01

系統事件：runCheck(projectPath, profilePath)

GRASP Pattern	套用對象	Rationale
Controller	:CheckCommand	接收 CLI 系統事件 runCheck(...)， 作為 UC-01 use case controller
Pure Fabrication	:ComplianceCheckService、:ProfileLoader、:JavaParserAdapter	為協調 use case 流程或封裝技術細節而建立的非 domain 物件
Polymorphism	rule:ComplianceRule + 4 Rule 子型別	4 種架構規則 檢查邏輯由 validate(files) 動態分派
Information Expert	:SuppressionStore	擁有抑制清單 持久化資訊
Low Coupling	:JavaParserAdapter、:SuppressionStore、:Reporter	以 port/adaptor 隔離第三方解 析器、持久化 及輸出細節， Service 不直接 依賴具體實作

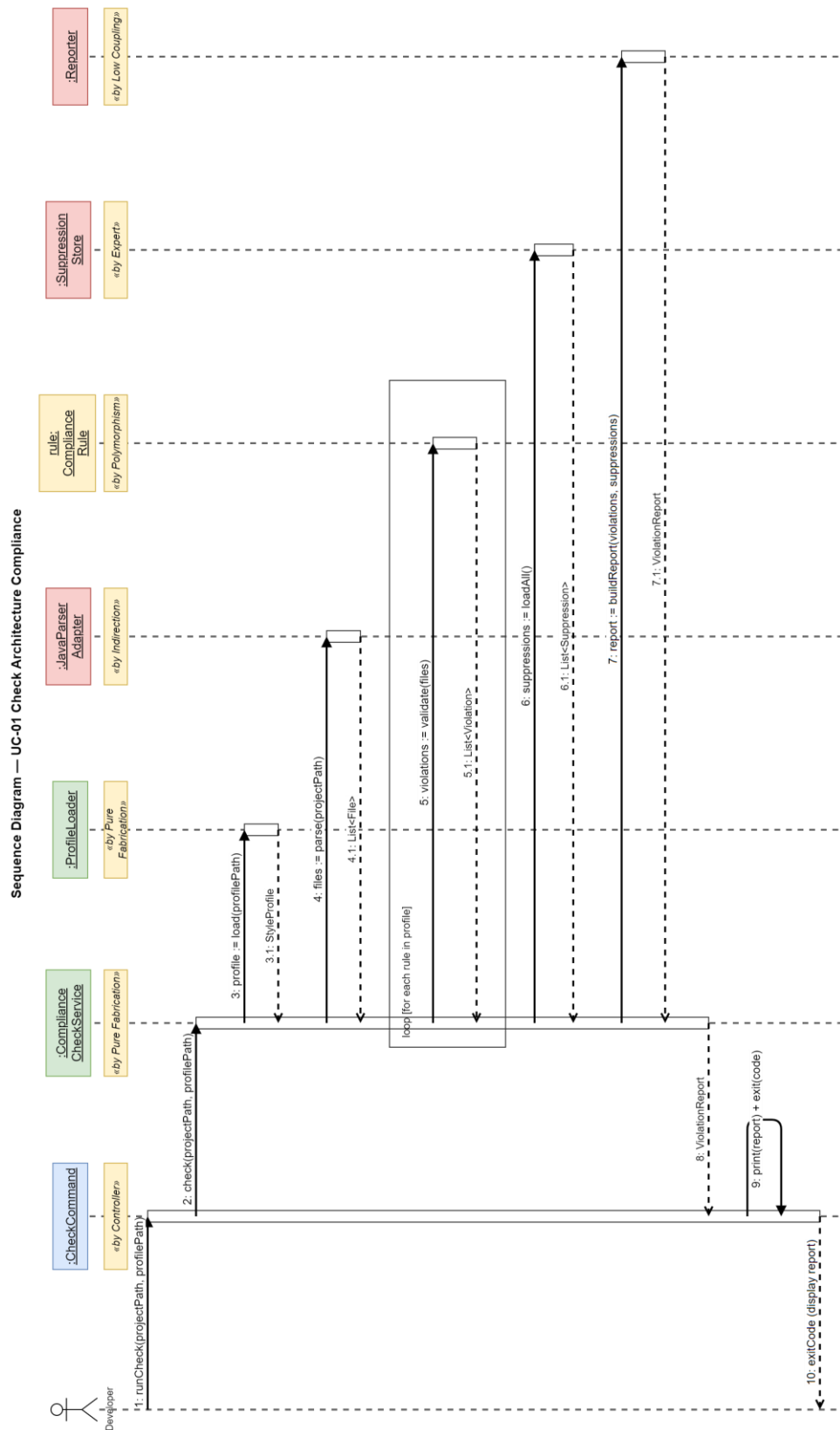


圖 3-2.2 SD for runCheck() (標示 GRASP 模式)

3.2.3 System Sequence Diagram — UC-02 Define Style Profile [new]

UC-02 為 Iteration II 最重要的新增使用案例。本節以 System Sequence Diagram

(SSD) 識別系統事件：initProfile(outputPath) 產生 YAML 範本檔供開發者編輯，Profile 的驗證與載入由 UC-05 (Load Style Profile) 負責，UC-02 不重複此職責。

System Sequence Diagram — UC-02 Define Style Profile

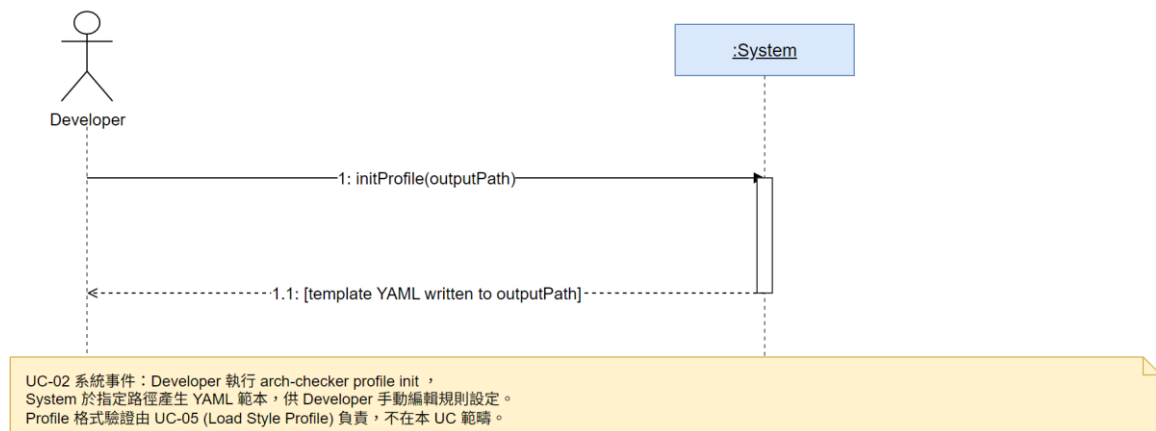


圖 3-2.3 UC-02 Define Style Profile — System Sequence Diagram

3.2.4 Sequence Diagrams with GRASP Patterns — UC-02 [new]

系統事件 1：initProfile(outputPath)

GRASP Pattern	套用對象	Rationale
Controller	:ProfileInitCommand	接收 initProfile(outputPath) 系統事件，作為 UC-02 use case controller。
Pure Fabrication	:ProfileTemplateService	Domain Model 無「產生設定檔範本」概念，為封裝 YAML 範本建立責任而建立。
Low Coupling	:ProfileInitCommand	直接組合 ProfileTemplateService，CLI 層不直接依賴 YAML 輸出細節。

Sequence Diagram — UC-02 System Event: initProfile(outputPath)

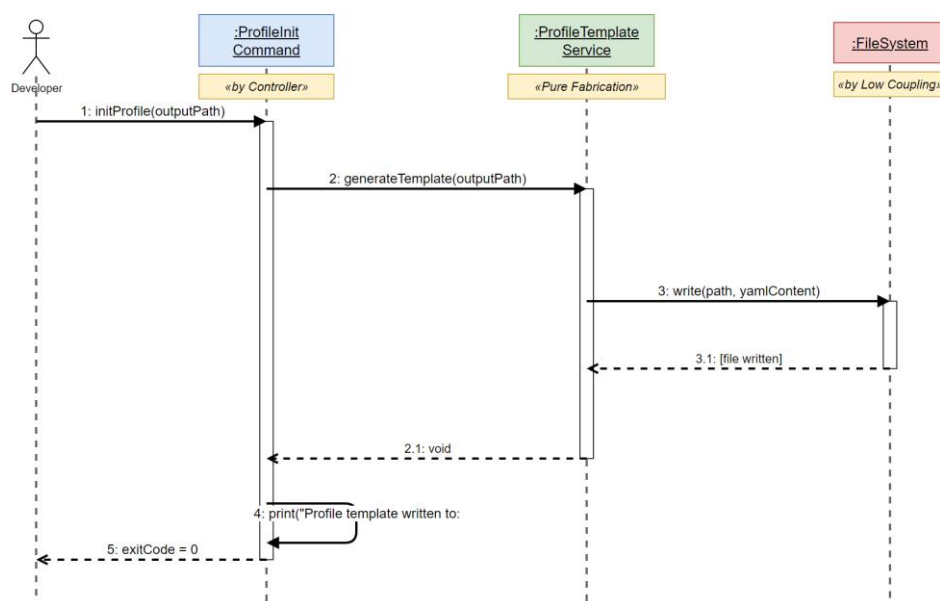


圖 3-2.4a SD for initProfile() (標示 GRASP 模式)

3.2.5 System Sequence Diagram — UC-03 Check with Interactive Fix Suggestions [new]

UC-03 為 Iteration II 新增之互動式修復使用案例。系統唯一入口事件為 fix(sourcePath, profilePath)；系統對每一筆違規呼叫 displayViolation(violation) 輸出資訊，Developer 逐一回覆 suppress(violationId) 觸發 UC-04 (<<include>>)，最後系統回傳 fixResult。

System Sequence Diagram — UC-03 Check with Interactive Fix Suggestions

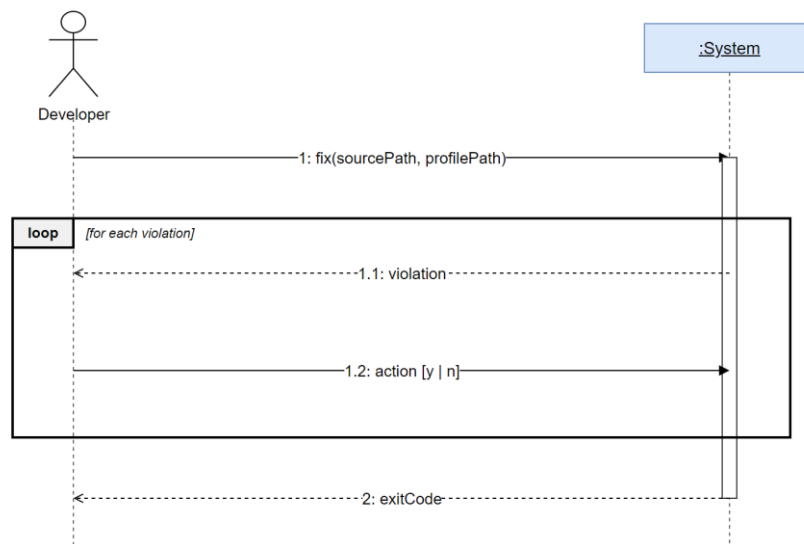


圖 3-2.5 UC-03 Check with Interactive Fix Suggestions — System Sequence Diagram

3.2.6 Sequence Diagram with GRASP Patterns — UC-03 [new]

系統事件：fix(sourcePath, profilePath)

GRASP 模式	套用類別	說明
Controller	FixCommand	作為 UC-03 之 use-case controller，接收並處理 fix 系統事件（一 use case 對應一 controller）。
Pure Fabrication	SuppressionService	封裝 suppress 完整流程（loadAll→建立 Suppression→save），不對應任何 Domain 概念；FixCommand 與 SuppressCommand 均可透過同一介面執行 suppress，達成複用（High Cohesion、Indirection）。
Low Coupling	FixCommand	FixCommand 僅依賴 ComplianceCheckService 與 SuppressionService 介面，I/O（包含 reason 輸入）直接走 System.in/out 而非耦合任何具體類別，便於替換實作。

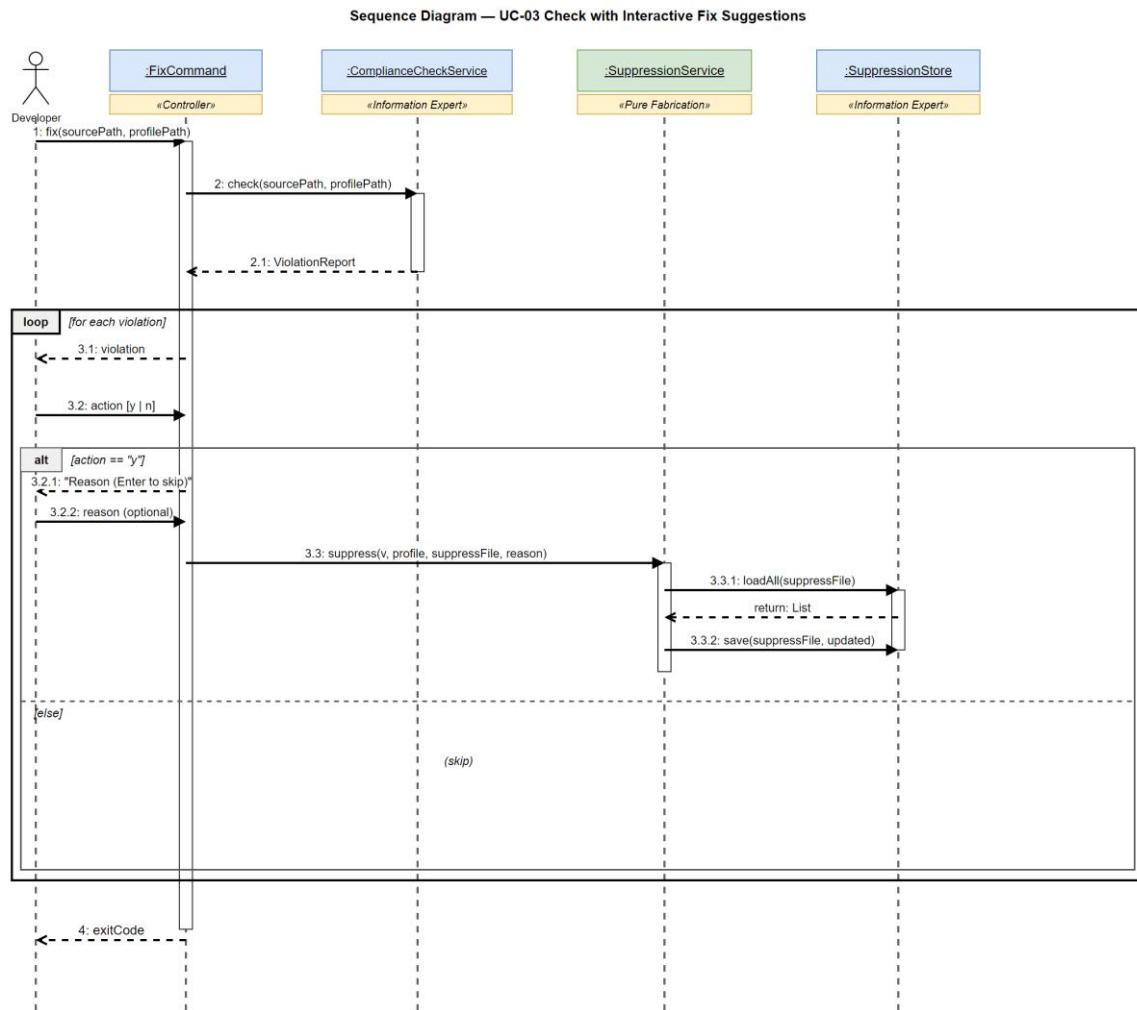
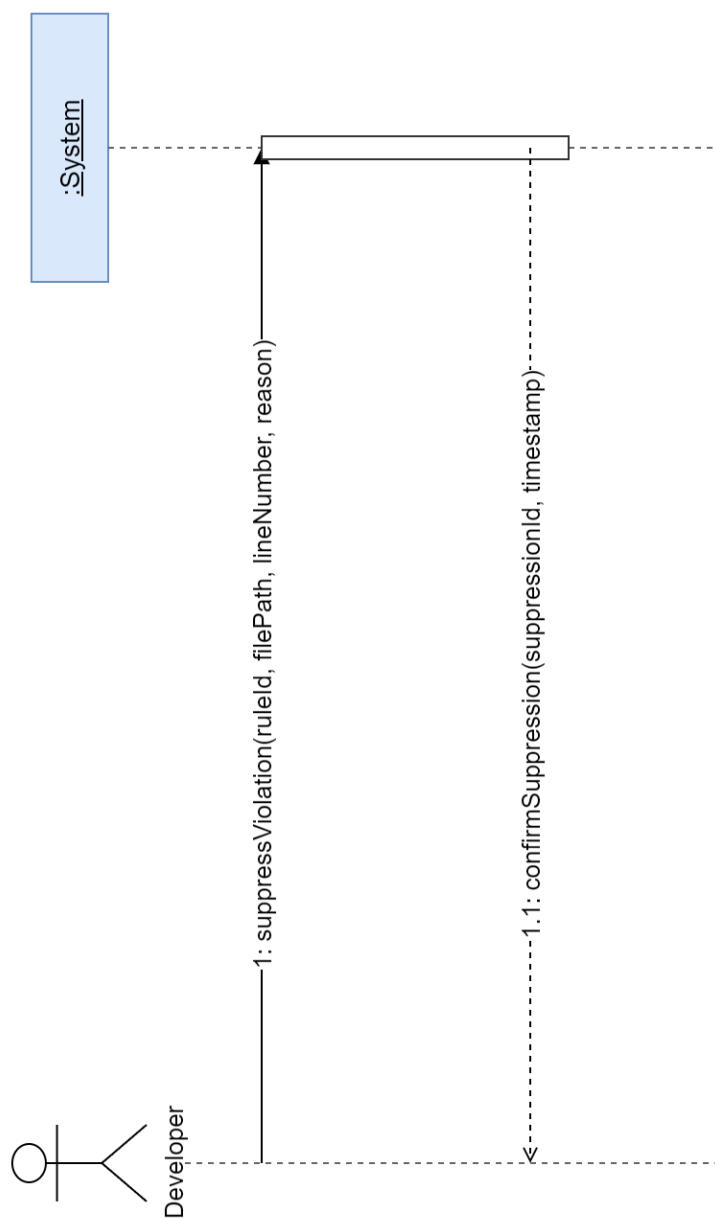


圖 3-2.6 SD for fix() (標示 GRASP 模式)

3.2.7 System Sequence Diagram — UC-04 Suppress a Violation

UC-04 為非互動式的單點抑制：Developer 直接對 :System 發出 suppress(constraintId, filePath, lineNumber, reason) 系統事件，系統持久化 Suppression 紀錄後回傳確認。

System Sequence Diagram — UC-04 Suppress a Violation



事件說明：Developer 對先前檢查中出現的違規執行非互動式抑制。
System 驗證輸入參數後，將 Suppression 記錄持久化至儲存庫，
並回傳確認訊息（含 suppressionId 與 timestamp）供 Developer 追蹤。

圖 3-2.7 UC-04 Suppress a Violation — System Sequence Diagram

3.2.8 Sequence Diagram with GRASP Patterns — UC-04

系統事件：suppress(constraintId, filePath, lineNumber, reason)

GRASP Pattern	套用對象	Rationale
Controller	:SuppressCommand	接收 suppress(...) 系統事件，作為 UC-04 use case controller
Pure Fabrication	:SuppressionStore	Domain Model 無「持久化抑制記錄」概念，為封裝 YAML 讀寫細節而建立
Information Expert	:SuppressionStore	擁有抑制清單持久化責任，掌握所有已存抑制記錄
Low Coupling	:SuppressCommand	僅依賴 SuppressionStore 介面，不直接耦合 YAML 輸出細節

Sequence Diagram — UC-04 Suppress a Violation

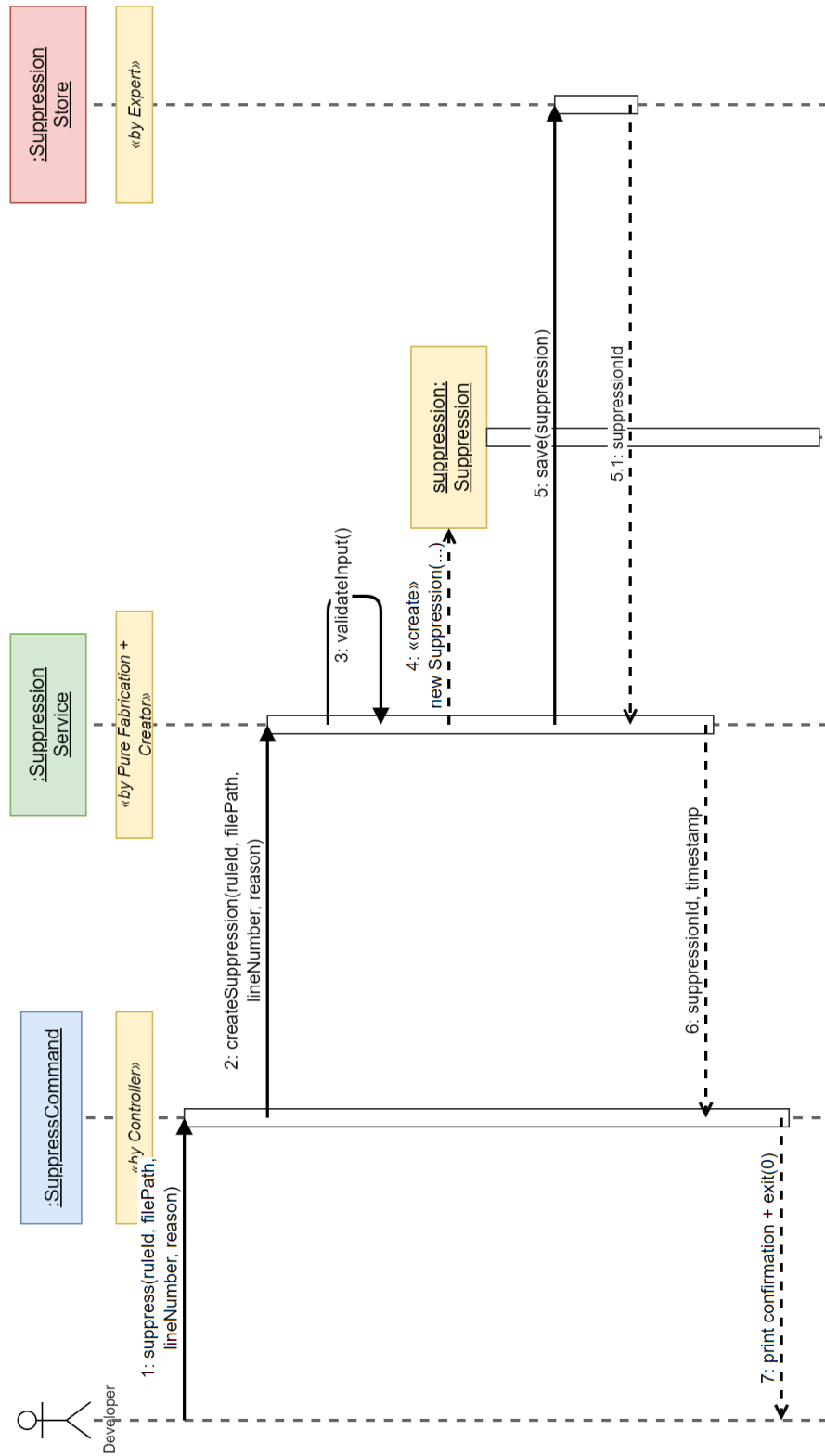


圖 3-2.8 SD for suppress() (標示 GRASP 模式)

3.3 Design Class Model

下圖為 Iteration II 之 Design Class Diagram (DCD)，涵蓋系統所有類別、屬性與主要方法。標示 [new] 者為新增類別，標示 [mod] 者為已修改之類別。四個 ComplianceRule 子類別 (NamingRule、DependencyRule、SupertypeRule、PackageRule) 在其產生的每個 Violation 上加入 suggestion 訊息，以支援 UC-03 的互動式修復顯示。

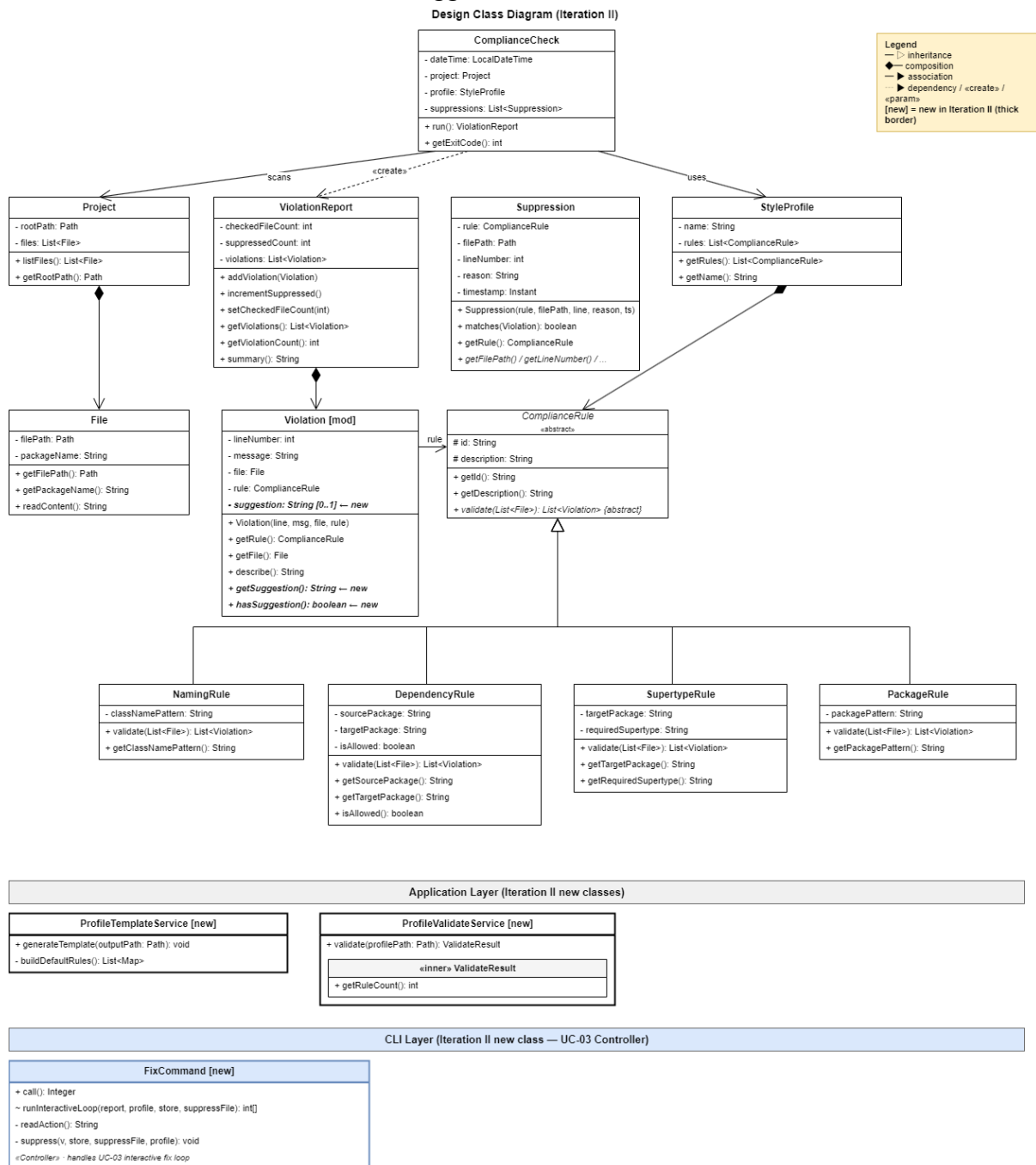


圖 3-3 Design Class Diagram (Iteration II ; [new] = 粗框新增)

4. Implementation Class Model

本章節依實作完成之 Java 程式碼繪製 Implementation Class Diagram (4.1)，與 §3.3 之 Design Class Model 進行差異比較 (4.2)，最後以 Lines of Code 量化實作規模 (4.3)。

4.1 Implementation Class Diagram

由於實作緊跟 DCD，ICD 在結構上與圖 3-3 相同。與 Iteration I 相比，主要新增為六個新類別（FixCommand、ProfileCommand、ProfileInitCommand、ProfileValidateCommand、ProfileTemplateService、ProfileValidateService）以及 Violation 上的 suggestion 欄位。UC-03 互動邏輯由 FixCommand 直接處理；suppress 操作委派 SuppressionService 執行，與 SuppressCommand 共用同一 Application 服務。

4.2 Differences between Design and Implementation Class Model

Table 4.2.1: Comparison of Design and Implementation (Iteration II changes)

下表逐一列出 Iteration II 新增或修改的所有類別與選定方法，比較 DCD（設計）與實際程式碼（實作）之差異。「Yes」表示該類別／方法存在於該成品中。

類別	方法 / 屬性	設計	實作
FixCommand [new]	call(): Integer	Yes	Yes
ProfileCommand [new]	run(): void	Yes	Yes
ProfileInitCommand [new]	call(): Integer	Yes	Yes
ProfileValidateCommand [new]	call(): Integer	Yes	Yes
FixCommand [new]	call(): Integer	Yes	Yes
FixCommand [new]	runInteractiveLoop(...): int[]	Yes	Yes
FixCommand [new]	readAction(): String	Yes	Yes
FixCommand [new]	suppress(violation, suppressFile): void	Yes	Yes
ProfileTemplateService [new]	generateTemplate(outputPath): void	Yes	Yes
ProfileValidateService [new]	validate(profilePath): ValidateResult	Yes	Yes
ProfileValidateService.ValidateResult [new]	getRuleCount(): int	Yes	Yes
Violation [mod]	suggestion: String [0..1]（新增屬性）	Yes	Yes
Violation [mod]	getSuggestion(): String（新增方法）	Yes	Yes
Violation [mod]	hasSuggestion(): boolean（新增方法）	Yes	Yes
NamingRule [mod]	validate(): 現於 Violation 上設定 suggestion	Yes	Yes
DependencyRule [mod]	validate(): 現於 Violation 上設定 suggestion	Yes	Yes
SupertypeRule [mod]	validate(): 現於 Violation 上設定 suggestion	Yes	Yes
PackageRule [mod]	validate(): 現於 Violation 上設定 suggestion	Yes	Yes
Main [mod]	subcommands: 新增 ProfileCommand、FixCommand	Yes	Yes

Table 4.2.2: Summary of Implementation Class/Method Changes

下表統計 Iteration II 整體之類別與方法變動數量。

	新增	移除	修改（簽章變更）
類別	7	0	6（Main、Violation、NamingRule、DependencyRule、SupertypeRule、PackageRule）
方法 / 屬性	17	0	0

4.3 Calculate Line of Code

Table 4.3.1: Line of Code of Classes

LOC 採「排除空行與單行 // 註解、整段 /* ... */ 區塊」之計算方式；method 數含 public、private、protected 之具體方法（不計 abstract 宣告與建構子，但計入介面方法）。Iteration II 新增之類別標示 [new]，修改之類別標示 [mod]。

No.	類別名稱	方法數	LOC（不含註解／空
-----	------	-----	------------

			行)
1	cli.Main	2	20
2	cli.CheckCommand	6	39
3	cli.SuppressCommand	8	39
4	cli.ProfileCommand [new]	2	11
5	cli.ProfileInitCommand [new]	3	18
6	cli.ProfileValidateCommand [new]	3	30
7	cli.FixCommand [new]	5	55
8	application.ComplianceCheckService	12	41
9	application.SuppressionService	6	39
10	cli.FixCommand [new]	15	87
11	application.ProfileTemplateService [new]	3	48
12	application.ProfileValidateService [new]	11	31
13	application.CodeParser (interface)	1	6
14	application.ProfileLoader (interface)	1	6
15	application.SuppressionStore (interface)	1	9
16	application.Reporter (interface)	1	5
17	domain.compliance.ComplianceCheck	13	45
18	domain.compliance.Violation [mod]	15	31
19	domain.compliance.ViolationReport	12	29
20	domain.compliance.Suppression	13	29
21	domain.codebase.File	7	24
22	domain.codebase.Project	6	18
23	domain.profile.StyleProfile	6	18
24	domain.rule.ComplianceRule	8	27
25	domain.rule.NamingRule [mod]	7	46
26	domain.rule.DependencyRule [mod]	10	52
27	domain.rule.SupertypeRule [mod]	8	44
28	domain.rule.PackageRule [mod]	5	36
29	infrastructure.parser.JavaParserAdapter	4	42
30	infrastructure.profile.YamlProfileLoader	3	69
31	infrastructure.report.ConsoleReporter	5	22
32	infrastructure.report.JsonReporter	6	36
33	infrastructure.suppression.YamlSuppressionStore	5	79
	合計	213	1131

5. Programming

本章節依需求 5.1 提供系統執行畫面截圖，並於 5.2 節錄 Iteration II 最具代表性之原始碼。

5.1 Snapshots of System Execution

5.1.1 UC-01 檢查專案是否合規執行畫面：

```
PS E:\Coding\arch-checker> arch-checker check demo/sample-project demo/demo-profile.yaml
demo\sample-project\src\main\java\com\example\service\PaymentManager.java:12 [R-NAME-01] Class 'P
.*(?:Manager|Helper|Util|Handler)$).*'
demo\sample-project\src\main\java\com\example\service\ChargeService.java:2 [R-DEP-01] Package 'co
example.controller' (import: com.example.controller.PaymentController)
demo\sample-project\src\main\java\com\example\domain\payment\CashPayment.java:12 [R-SUP-01] Class
nt'
demo\sample-project\src\main\java\com\example\controller\PaymentController.java:0 [R-PKG-01] Requ
y' not present in the project
--
Checked 7 file(s); 4 violation(s); 0 suppressed.
```

圖 5.1 UC-01 指令

5.1.2 UC-02 profile init 執行畫面：

1) 輸入指令與預期輸出：

```
PS E:\Coding\arch-checker> arch-checker profile init demo/my-profile.yaml
Profile template written to: demo\my-profile.yaml
```

圖 5.2 UC-02 指令

2) 拿到樣板 Profile:

```
1      # arch-checker Style Profile
2      # See https://github.com/your-org/arch-checker for full documentation.
3      name: my-profile
4      rules:
5          # NamingRule: enforce a class name regex pattern
6          - id: R-NAME-01
7            type: naming
8            description: "Service classes must end with 'Service'"
9            classNamePattern: ".*Service"
10
11         # DependencyRule: control allowed/forbidden package dependencies
12         - id: R-DEP-01
13           type: dependency
14           description: "Domain must not depend on infrastructure"
15           sourcePackage: "**.domain.**"
16           targetPackage: "**.infrastructure.**"
17           isAllowed: false
18
19         # SupertypeRule: require classes in a package to extend/implement a type
20         - id: R-SUP-01
21           type: supertype
22           description: "All rule classes must extend ComplianceRule"
23           targetPackage: "**.domain.rule"
24           requiredSupertype: "ComplianceRule"
25
26         # PackageRule: assert a package pattern exists in the project
27         - id: R-PKG-01
28           type: package
29           description: "Domain layer must exist"
30           packagePattern: "**.domain.**"
31
```

圖 5.3 產生的樣板 Profile

5.1.3 UC-03 互動式修正違規

1) 指令執行與互動修正違規:

```
PS E:\Coding\arch-checker> arch-checker fix demo/sample-project demo/demo-profile.yaml --suppress-file demo/.arch-checker-suppress.yaml

Violation 1/4:
demo\sample-project\src\main\java\com\example\service\PaymentManager.java:12 [R-NAME-01] Class 'PaymentManager' violates
naming pattern '(?!.*(?:Manager|Helper|Util|Handler)$).*'
Suggestion: Rename 'PaymentManager' to match pattern '(?!.*(?:Manager|Helper|Util|Handler)$).*'
Suppress? [y/n/q]: y
Reason (Enter to skip): keep it

Violation 2/4:
demo\sample-project\src\main\java\com\example\service\ChargeService.java:2 [R-DEP-01] Package 'com.example.service' must
not depend on 'com.example.controller' (import: com.example.controller.PaymentController)
Suggestion: Remove import 'com.example.controller.PaymentController'
Suppress? [y/n/q]: n

Violation 3/4:
demo\sample-project\src\main\java\com\example\domain\payment\CashPayment.java:12 [R-SUP-01] Class 'CashPayment' must
extend/implement 'Payment'
Suggestion: Add 'extends Payment' or 'implements Payment' to class declaration
Suppress? [y/n/q]: n

Violation 4/4:
demo\sample-project\src\main\java\com\example\controller\PaymentController.java:0 [R-PKG-01] Required package pattern
'com.example.repository' not present in the project
Suggestion: Create at least one class in a package matching 'com.example.repository'
Suppress? [y/n/q]: y
Reason (Enter to skip): already switch the package structure

--
Checked 7 file(s); 2 violation(s) remaining; 2 suppressed.
```

圖 5.4 UC-03 指令執行畫面

5.1.4 UC-04 抑制一個違規

```
PS E:\Coding\arch-checker> arch-checker suppress
demo/demo-profile.yaml
R-NAME-01
demo/sample-project/src/main/java/com/example/service/PaymentManager.java
12
'legacy name; refactor in v2'
--suppress-file demo/.arch-checker-suppress.yaml

Suppressed: R-NAME-01 at demo\sample-project\src\main\java\com\example\service\PaymentManager.java:12
```

圖 5.5 UC-04 指令執行畫面

5.2 Source Code Listing

下列依序列出 Iteration II 最重要的原始碼（新增或修改之類別）。完整原始碼請參見專案儲存庫。

— Violation.java [mod]（新增 suggestion 欄位）

```
package com.archchecker.domain.compliance;

import com.archchecker.domain.codebase.File;
import com.archchecker.domain.rule.ComplianceRule;

public class Violation {
    private final int lineNumber;
    private final String message;
    private final File file;
    private final ComplianceRule rule;
    private final String suggestion; // [0..1]

    public Violation(int lineNumber, String message, File file, ComplianceRule rule) {
        this(lineNumber, message, file, rule, null);
    }

    public Violation(int lineNumber, String message, File file,
        ComplianceRule rule, String suggestion) {
```

```

        this.lineNumber = lineNumber;
        this.message = message;
        this.file = file;
        this.rule = rule;
        this.suggestion = suggestion;
    }

    public int getLineNumber() { return lineNumber; }
    public String getMessage() { return message; }
    public File getFile() { return file; }
    public ComplianceRule getRule() { return rule; }
    public String getSuggestion() { return suggestion; }
    public boolean hasSuggestion() { return suggestion != null && !suggestion.isBlank(); }

    public String describe() {
        return file.getPath() + ":" + lineNumber
            + " [" + rule.getId() + "] " + message;
    }
}

```

— InteractiveFixer.java [deleted — logic moved to FixCommand]

```

package com.archchecker.application;

import com.archchecker.domain.compliance.Suppression;
import com.archchecker.domain.compliance.Violation;
import com.archchecker.domain.compliance.ViolationReport;
import com.archchecker.domain.profile.StyleProfile;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.PrintStream;
import java.nio.file.Path;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;

public class InteractiveFixer {
    private final SuppressionStore store;
    private final PrintStream out;
    private final BufferedReader in;

    public InteractiveFixer(SuppressionStore store, PrintStream out, BufferedReader in) {
        this.store = store;
        this.out = out;
        this.in = in;
    }

    public FixResult fix(ViolationReport report, Path suppressionFile, StyleProfile profile) {
        List<Violation> violations = new ArrayList<>(report.getViolations());
        int newlySuppressed = 0;
        List<Violation> remaining = new ArrayList<>();
        boolean quit = false;

        for (int i = 0; i < violations.size(); i++) {
            Violation v = violations.get(i);
            if (quit) {
                remaining.add(v);
                continue;
            }
            out.println();
            out.println("Violation " + (i + 1) + "/" + violations.size() + ":");
            out.println("  " + v.describe());
            if (v.hasSuggestion()) {
                out.println("    Suggestion: " + v.getSuggestion());
            }
            String answer = prompt();
            switch (answer) {
                case "y" -> {
                    suppress(v, suppressionFile, profile);
                    newlySuppressed++;
                }
            }
        }
    }
}

```

```

        }
        case "q" -> {
            remaining.add(v);
            quit = true;
        }
        default -> remaining.add(v);
    }
}
return new FixResult(newlySuppressed, remaining);
}

private String prompt() {
    while (true) {
        out.print("Suppress? [y/n/q]: ");
        try {
            String line = in.readLine();
            if (line == null) return "q";
            String trimmed = line.trim().toLowerCase();
            if (trimmed.equals("y") || trimmed.equals("yes")) return "y";
            if (trimmed.equals("n") || trimmed.equals("no")) return "n";
            if (trimmed.equals("q") || trimmed.equals("quit")) return "q";
            out.print("Invalid input. Please enter y, n, or q: ");
        } catch (IOException e) {
            return "q";
        }
    }
}

private void suppress(Violation v, Path suppressionFile, StyleProfile profile) {
    List<Suppression> all = new ArrayList<>(store.loadAll(suppressionFile, profile));
    all.add(new Suppression(v.getRule(), v.getFile().getFilePath(),
        v.getLineNumber(), "Suppressed interactively", Instant.now()));
    store.save(suppressionFile, all);
}

public static class FixResult {
    private final int newlySuppressed;
    private final List<Violation> remaining;

    public FixResult(int newlySuppressed, List<Violation> remaining) {
        this.newlySuppressed = newlySuppressed;
        this.remaining = remaining;
    }

    public int getNewlySuppressed() { return newlySuppressed; }
    public List<Violation> getRemaining() { return remaining; }
    public int getExitCode() { return remaining.isEmpty() ? 0 : 1; }
}
}

```

— ProfileTemplateService.java [new]

```
package com.archchecker.application;
```

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
```

```
public class ProfileTemplateService {
```

```
    private static final String TEMPLATE =
        "# arch-checker Style Profile\n"
        + "# See https://github.com/your-org/arch-checker for full documentation.\n"
        + "name: my-profile\n"
        + "rules:\n"
        + "  # NamingRule: enforce a class name regex pattern\n"
        + "  - id: R-NAME-01\n"
        + "    type: naming\n"
        + "    description: \"Service classes must end with 'Service'\"\n"
        + "    classNamePattern: \".*Service\"";

```



```

+ "\n"
+ " # DependencyRule: control allowed/forbidden package dependencies\n"
+ " - id: R-DEP-01\n"
+ "   type: dependency\n"
+ "   description: \"Domain must not depend on infrastructure\"\n"
+ "   sourcePackage: \"***.domain.**\"\n"
+ "   targetPackage: \"***.infrastructure.**\"\n"
+ "   isAllowed: false\n"
+ "\n"
+ " # SupertypeRule: require classes in a package to extend/implement a type\n"
+ " - id: R-SUP-01\n"
+ "   type: supertype\n"
+ "   description: \"All rule classes must extend ComplianceRule\"\n"
+ "   targetPackage: \"***.domain.rule\"\n"
+ "   requiredSupertype: \"ComplianceRule\"\n"
+ "\n"
+ " # PackageRule: assert a package pattern exists in the project\n"
+ " - id: R-PKG-01\n"
+ "   type: package\n"
+ "   description: \"Domain layer must exist\"\n"
+ "   packagePattern: \"***.domain.**\"\n";

public void generateTemplate(Path outputPath) {
    try {
        if (outputPath.getParent() != null) {
            Files.createDirectories(outputPath.getParent());
        }
        Files.writeString(outputPath, TEMPLATE, StandardCharsets.UTF_8);
    } catch (IOException e) {
        throw new RuntimeException("Failed to write profile template: " + e.getMessage(),
e);
    }
}
}

```

— ProfileValidateService.java [new]

```
package com.archchecker.application;
```

```
import com.archchecker.domain.profile.StyleProfile;
import com.archchecker.domain.rule.ComplianceRule;
```

```
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
```

```
public class ProfileValidateService {
    private final ProfileLoader profileLoader;

    public ProfileValidateService(ProfileLoader profileLoader) {
        this.profileLoader = profileLoader;
    }

    public ValidationResult validate(Path profilePath) {
        StyleProfile profile = profileLoader.load(profilePath);
        List<String> ruleIds = new ArrayList<>();
        for (ComplianceRule r : profile.getRules()) {
            ruleIds.add(r.getId() + " (" + r.getClass().getSimpleName() + ")");
        }
        return new ValidationResult(profile.getName(), ruleIds);
    }

    public static class ValidationResult {
        private final String profileName;
        private final List<String> ruleDescriptions;

        public ValidationResult(String profileName, List<String> ruleDescriptions) {
            this.profileName = profileName;
            this.ruleDescriptions = ruleDescriptions;
        }
    }
}

```

```

        public String getProfileName() { return profileName; }
        public int getRuleCount() { return ruleDescriptions.size(); }
        public List<String> getRuleDescriptions() { return ruleDescriptions; }
    }
}

```

— FixCommand.java [new]

```

package com.archchecker.cli;

import com.archchecker.application.ComplianceCheckService;
import com.archchecker.application.SuppressionStore;
import com.archchecker.domain.compliance.ViolationReport;
import com.archchecker.domain.profile.StyleProfile;
import com.archchecker.infrastructure.parser.JavaParserAdapter;
import com.archchecker.infrastructure.profile.YamlProfileLoader;
import com.archchecker.infrastructure.report.ConsoleReporter;
import com.archchecker.infrastructure.suppression.YamlSuppressionStore;
import picocli.CommandLine.Command;
import picocli.CommandLine.Option;
import picocli.CommandLine.Parameters;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.nio.file.Path;
import java.util.concurrent.Callable;

@Command(name = "fix",
        description = "Check architecture compliance with interactive fix suggestions.")
public class FixCommand implements Callable<Integer> {

    @Parameters(index = "0", description = "Java project root directory.")
    private Path projectPath;

    @Parameters(index = "1", description = "Style Profile YAML file.")
    private Path profilePath;

    @Option(names = {"-s", "--suppress-file"},
            description = "Suppression file (default: .arch-checker-suppress.yaml).",
            defaultValue = ".arch-checker-suppress.yaml")
    private Path suppressionFile;

    @Override
    public Integer call() {
        YamlProfileLoader profileLoader = new YamlProfileLoader();
        YamlSuppressionStore store = new YamlSuppressionStore();

        ComplianceCheckService checkService = new ComplianceCheckService(
            new JavaParserAdapter(), profileLoader, store);
        ComplianceCheckService.CheckResult result =
            checkService.run(projectPath, profilePath, suppressionFile);

        ViolationReport report = result.getReport();
        if (report.getViolationCount() == 0) {
            new ConsoleReporter().render(report);
            return result.getExitCode();
        }

        StyleProfile profile = profileLoader.load(profilePath);
        int[] counts = runInteractiveLoop(report, profile, store, suppressionFile);
        int newlySuppressed = counts[0];
        int remaining = counts[1];
        out.println();
        out.println("--");

        out.println("Checked " + report.getCheckedFileCount() + " file(s); "
            + remaining + " violation(s) remaining; "
            + (report.getSuppressedCount() + newlySuppressed) + " suppressed.");
    }
}

```

```

        return remaining == 0 ? 0 : 1;
    }
}

```

— ProfileInitCommand.java [new]

```

package com.archchecker.cli;

import com.archchecker.application.ProfileTemplateService;
import picocli.CommandLine.Command;
import picocli.CommandLine.Parameters;

import java.nio.file.Path;
import java.util.concurrent.Callable;

@Command(name = "init",
        description = "Generate a Style Profile YAML template at the specified path.")
public class ProfileInitCommand implements Callable<Integer> {

    @Parameters(index = "0", description = "Output path for the generated YAML template.")
    private Path outputPath;

    @Override
    public Integer call() {
        new ProfileTemplateService().generateTemplate(outputPath);
        System.out.println("Profile template written to: " + outputPath);
        return 0;
    }
}

```

— ProfileValidateCommand.java [new]

```

package com.archchecker.cli;

import com.archchecker.application.ProfileValidateService;
import com.archchecker.infrastructure.profile.YamlProfileLoader;
import picocli.CommandLine.Command;
import picocli.CommandLine.Parameters;

import java.nio.file.Path;
import java.util.concurrent.Callable;

@Command(name = "validate",
        description = "Validate a Style Profile YAML and display loaded rules.")
public class ProfileValidateCommand implements Callable<Integer> {

    @Parameters(index = "0", description = "Style Profile YAML file to validate.")
    private Path profilePath;

    @Override
    public Integer call() {
        ProfileValidateService service =
            new ProfileValidateService(new YamlProfileLoader());
        try {
            ProfileValidateService.ValidateResult result = service.validate(profilePath);
            System.out.println("Profile '" + result.getProfileName()
                + "' is valid. Loaded " + result.getRuleCount() + " rule(s):");
            for (String desc : result.getRuleDescriptions()) {
                System.out.println("  - " + desc);
            }
            return 0;
        } catch (Exception e) {
            System.err.println("Validation failed: " + e.getMessage());
            return 2;
        }
    }
}

```

6. Unit Testing

6.1 Snapshots of Testing Result

以下為 mvn test 執行結果截圖，共 29 個 JUnit 5 測試全數通過：

6.2 Unit Testing Code Listing

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.archchecker.application.ComplianceCheckServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.056 s -- in com.archchecker.application.ComplianceCheckServiceTest
[INFO] Running com.archchecker.application.ProfileTemplateServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.117 s -- in com.archchecker.application.ProfileTemplateServiceTest
[INFO] Running com.archchecker.application.SuppressionServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in com.archchecker.application.SuppressionServiceTest
[INFO] Running com.archchecker.cli.FixCommandTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.017 s -- in com.archchecker.cli.FixCommandTest
[INFO] Running com.archchecker.domain.rule.DependencyRuleTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.018 s -- in com.archchecker.domain.rule.DependencyRuleTest
[INFO] Running com.archchecker.domain.rule.NamingRuleTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.019 s -- in com.archchecker.domain.rule.NamingRuleTest
[INFO] Running com.archchecker.domain.rule.PackageRuleTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in com.archchecker.domain.rule.PackageRuleTest
[INFO] Running com.archchecker.domain.rule.SupertypeRuleTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 s -- in com.archchecker.domain.rule.SupertypeRuleTest
[INFO] Running com.archchecker.infrastructure.profile.YamlProfileLoaderTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.015 s -- in com.archchecker.infrastructure.profile.YamlProfileLoaderTest
[INFO] Running com.archchecker.infrastructure.suppression.YamlSuppressionStoreTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.027 s -- in com.archchecker.infrastructure.suppression.YamlSuppressionStoreTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 29, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

下列測試檔案涵蓋 Iteration II 的新增功能：

— FixCommandTest.java [new] (UC-03 互動式修復)

```
package com.archchecker.cli;

import com.archchecker.domain.codebase.File;
import com.archchecker.domain.compliance.Suppression;
import com.archchecker.domain.compliance.Violation;
import com.archchecker.domain.compliance.ViolationReport;
import com.archchecker.domain.profile.StyleProfile;
import com.archchecker.domain.rule.ComplianceRule;
import org.junit.jupiter.api.Test;

import java.io.BufferedReader;
import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import java.io.StringReader;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class FixCommandTest {

    @Test
    void actionYSuppressesViolationAndReturnsExitZero() {
        File file = new File(Paths.get("/x/A.java"), "com.foo");
        StubRule rule = new StubRule("R-NAME-01");
        Violation v = new Violation(1, "bad name", file, rule, "Rename it");
        ViolationReport report = reportWith(v);
        SpyStore store = new SpyStore();
        StyleProfile profile = new StyleProfile("p", List.of(rule));
```

```

        int[] counts = fixCommandWith("y\n", store)
            .runInteractiveLoop(report, profile, store, Paths.get("/s.yaml"));

        assertEquals(1, counts[0], "newlySuppressed");
        assertEquals(0, counts[1], "remaining");
        assertEquals(1, store.saved.size());
        assertEquals(1, store.saved.size());
    }

    @Test
    void actionNKeepsViolationInRemaining() {
        File file = new File(Paths.get("/x/A.java"), "com.foo");
        StubRule rule = new StubRule("R-NAME-01");
        Violation v = new Violation(1, "bad name", file, rule);
        ViolationReport report = reportWith(v);
        SpyStore store = new SpyStore();

        int[] counts = fixCommandWith("n\n", store)
            .runInteractiveLoop(report, new StyleProfile("p", List.of(rule)), store,
Paths.get("/s.yaml"));
        assertEquals(0, counts[0], "newlySuppressed");

        assertEquals(1, counts[1], "remaining");
        assertTrue(store.saved.isEmpty());
    }

    @Test
    void actionQStopsLoopAndKeepsAllRemainingViolations() {

        File file = new File(Paths.get("/x/A.java"), "com.foo");
        StubRule rule = new StubRule("R-NAME-01");
        Violation v1 = new Violation(1, "v1", file, rule);
        Violation v2 = new Violation(2, "v2", file, rule);
        SpyStore store = new SpyStore();

        int[] counts = fixCommandWith("q\n", store)
            .runInteractiveLoop(reportWith(v1, v2), new StyleProfile("p", List.of(rule)),
store, Paths.get("/s.yaml"));

        assertEquals(2, counts[1], "both violations remain after quit");
        assertEquals(0, counts[0]);
    }

    @Test
    void invalidInputPromptsAgainBeforeAccepting() {
        File file = new File(Paths.get("/x/A.java"), "com.foo");

        StubRule rule = new StubRule("R-NAME-01");
        Violation v = new Violation(1, "bad", new File(Paths.get("/x/A.java"), "com.foo"),
rule);
        SpyStore store = new SpyStore();
        ByteArrayOutputStream buf = new ByteArrayOutputStream();

        FixCommand cmd = new FixCommand(new PrintStream(buf),
            new BufferedReader(new StringReader("maybe\ny\n")));

        cmd.runInteractiveLoop(reportWith(v), new StyleProfile("p", List.of(rule)), store,
Paths.get("/s.yaml"));

        assertTrue(buf.toString().contains("Invalid input"),

            "should re-prompt on invalid input");
    }

```

```

private static FixCommand fixCommandWith(String input, SpyStore store) {
    ByteArrayOutputStream buf = new ByteArrayOutputStream();
    return new FixCommand(new PrintStream(buf),
        new BufferedReader(new StringReader(input)));
}

private static ViolationReport reportWith(Violation... violations) {
    ViolationReport r = new ViolationReport();
    r.setCheckedFileCount(1);
    for (Violation v : violations) r.addViolation(v);
    return r;
}

private static class StubRule extends ComplianceRule {
    StubRule(String id) { super(id, "stub"); }
    @Override public List<Violation> validate(List<File> files) {
        return Collections.emptyList();
    }
}

private static class SpyStore implements SuppressionStore {
    final List<Suppression> saved = new ArrayList<>();
    @Override public List<Suppression> loadAll(Path f, StyleProfile p) { return saved; }
    @Override public void save(Path f, List<Suppression> s) {
        saved.clear(); saved.addAll(s);
    }
}
}

```

— ProfileTemplateServiceTest.java [new] (UC-02 範本產生)

```

package com.archchecker.application;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

import static org.junit.jupiter.api.Assertions.*;

class ProfileTemplateServiceTest {

    @Test
    void generateTemplateCreatesReadableYamlFile(@TempDir Path tmp) throws IOException {
        Path output = tmp.resolve("arch.yaml");
        ProfileTemplateService service = new ProfileTemplateService();

        service.generateTemplate(output);

        assertTrue(Files.exists(output), "template file should be created");
        String content = Files.readString(output);
        assertTrue(content.contains("name:"), "template must include 'name:' field");
        assertTrue(content.contains("rules:"), "template must include 'rules:' section");
    }

    @Test
    void generatedTemplateContainsAllFourRuleTypes(@TempDir Path tmp) throws IOException {
        Path output = tmp.resolve("arch.yaml");
        new ProfileTemplateService().generateTemplate(output);

        String content = Files.readString(output);
        assertTrue(content.contains("type: naming"), "must include naming rule example");
        assertTrue(content.contains("type: dependency"), "must include dependency rule example");
        assertTrue(content.contains("type: supertype"), "must include supertype rule example");
    }
}

```

```

        assertTrue(content.contains("type: package"), "must include package rule example");
    }

    @Test
    void generatedTemplateIsLoadableByYamlProfileLoader(@TempDir Path tmp) throws IOException {
        Path output = tmp.resolve("arch.yaml");
        new ProfileTemplateService().generateTemplate(output);

        com.archchecker.infrastructure.profile.YamlProfileLoader loader =
            new com.archchecker.infrastructure.profile.YamlProfileLoader();
        com.archchecker.domain.profile.StyleProfile profile = loader.load(output);

        assertNotNull(profile);
        assertEquals(4, profile.getRules().size(), "template must produce 4 rules");
    }
}

```

— ComplianceCheckServiceTest.java（核心合規檢查）

```

package com.archchecker.application;

import com.archchecker.domain.codebase.File;
import com.archchecker.domain.codebase.Project;
import com.archchecker.domain.compliance.Suppression;
import com.archchecker.domain.compliance.Violation;
import com.archchecker.domain.rule.ComplianceRule;
import com.archchecker.domain.profile.StyleProfile;
import org.junit.jupiter.api.Test;

import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.Instant;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class ComplianceCheckServiceTest {

    @Test
    void runReturnsExitZeroWhenNoViolations() {
        FakeParser parser = new FakeParser(new Project(Paths.get("/x"),
            List.of(new File(Paths.get("/x/A.java"), "com.foo"))));
        FakeProfileLoader profileRepo = new FakeProfileLoader(
            new StyleProfile("p", List.of(new AlwaysPassRule())));
        FakeSupStore supRepo = new FakeSupStore(Collections.emptyList());
        ComplianceCheckService service = new ComplianceCheckService(parser, profileRepo,
supRepo);

        ComplianceCheckService.CheckResult r = service.run(
            Paths.get("/x"), Paths.get("/p.yaml"), Paths.get("/s.yaml"));

        assertEquals(0, r.getExitCode());
        assertEquals(0, r.getReport().getViolationCount());
        assertEquals(1, r.getReport().getCheckedFileCount());
    }

    @Test
    void runReturnsExitOneWhenViolationsFound() {
        File file = new File(Paths.get("/x/A.java"), "com.foo");
        FakeParser parser = new FakeParser(new Project(Paths.get("/x"), List.of(file)));
        AlwaysFailRule rule = new AlwaysFailRule(file);
        FakeProfileLoader profileRepo = new FakeProfileLoader(
            new StyleProfile("p", List.of(rule)));
        FakeSupStore supRepo = new FakeSupStore(Collections.emptyList());
        ComplianceCheckService service = new ComplianceCheckService(parser, profileRepo,
supRepo);
    }
}

```

```

        ComplianceCheckService.CheckResult r = service.run(
            Paths.get("/x"), Paths.get("/p.yaml"), Paths.get("/s.yaml"));

        assertEquals(1, r.getExitCode());
        assertEquals(1, r.getReport().getViolationCount());
    }

    @Test
    void suppressedViolationIsExcludedFromReport() {
        File file = new File(Paths.get("/x/A.java"), "com.foo");
        FakeParser parser = new FakeParser(new Project(Paths.get("/x"), List.of(file)));
        AlwaysFailRule rule = new AlwaysFailRule(file);
        Suppression sup = new Suppression(rule, file.getFilePath(), 1, "ack", Instant.now());
        FakeProfileLoader profileRepo = new FakeProfileLoader(
            new StyleProfile("p", List.of(rule)));
        FakeSupStore supRepo = new FakeSupStore(List.of(sup));
        ComplianceCheckService service = new ComplianceCheckService(parser, profileRepo,
supRepo);

        ComplianceCheckService.CheckResult r = service.run(
            Paths.get("/x"), Paths.get("/p.yaml"), Paths.get("/s.yaml"));

        assertEquals(0, r.getExitCode());
        assertEquals(0, r.getReport().getViolationCount());
        assertEquals(1, r.getReport().getSuppressedCount());
    }

    private static class FakeParser implements CodeParser {
        private final Project project;
        FakeParser(Project p) { this.project = p; }
        @Override public Project parseProject(Path rootPath) { return project; }
    }

    private static class FakeProfileLoader implements ProfileLoader {
        private final StyleProfile profile;
        FakeProfileLoader(StyleProfile p) { this.profile = p; }
        @Override public StyleProfile load(Path profilePath) { return profile; }
    }

    private static class FakeSupStore implements SuppressionStore {
        private final List<Suppression> stored;
        FakeSupStore(List<Suppression> s) { this.stored = new ArrayList<>(s); }
        @Override public List<Suppression> loadAll(Path f, StyleProfile p) { return stored; }
        @Override public void save(Path f, List<Suppression> s) { stored.clear();
stored.addAll(s); }
    }

    private static class AlwaysPassRule extends ComplianceRule {
        AlwaysPassRule() { super("PASS", "always pass"); }
        @Override public List<Violation> validate(List<File> files) {
            return Collections.emptyList();
        }
    }

    private static class AlwaysFailRule extends ComplianceRule {
        private final File reportedFile;
        AlwaysFailRule(File f) { super("FAIL", "always fail"); this.reportedFile = f; }
        @Override public List<Violation> validate(List<File> files) {
            return List.of(new Violation(1, "synthetic violation", reportedFile, this));
        }
    }
}

```

— NamingRuleTest.java (NamingRule 含 suggestion)

```

package com.archchecker.domain.rule;

import com.archchecker.domain.codebase.File;

```



```

import com.archchecker.domain.compliance.Violation;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class NamingRuleTest {

    @Test
    void compliantClassNameProducesNoViolation(@TempDir Path tmp) throws IOException {
        Path src = writeJava(tmp, "AccountService",
            "package com.foo;\n\nclass AccountService {}\n");
        NamingRule rule = new NamingRule("R-NAME-01", "Service suffix", ".*Service");

        List<Violation> result = rule.validate(List.of(new File(src, "com.foo")));

        assertTrue(result.isEmpty(), "expected no violation but got " + result);
    }

    @Test
    void nonCompliantClassNameProducesOneViolation(@TempDir Path tmp) throws IOException {
        Path src = writeJava(tmp, "AccountManager",
            "package com.foo;\n\nclass AccountManager {}\n");
        NamingRule rule = new NamingRule("R-NAME-01", "Service suffix", ".*Service");

        List<Violation> result = rule.validate(List.of(new File(src, "com.foo")));

        assertEquals(1, result.size());
        assertEquals(2, result.get(0).getLineNumber());
        assertTrue(result.get(0).getMessage().contains("AccountManager"));
    }

    @Test
    void multipleClassesInOneFileAreEachChecked(@TempDir Path tmp) throws IOException {
        Path src = writeJava(tmp, "Mixed",
            "package com.foo;\n\nclass GoodService {}\n\nclass BadName {}\n");
        NamingRule rule = new NamingRule("R-NAME-01", "Service suffix", ".*Service");

        List<Violation> result = rule.validate(List.of(new File(src, "com.foo")));

        assertEquals(1, result.size());
        assertTrue(result.get(0).getMessage().contains("BadName"));
    }

    private static Path writeJava(Path dir, String name, String body) throws IOException {
        Path p = dir.resolve(name + ".java");
        Files.writeString(p, body);
        return p;
    }
}

```

— DependencyRuleTest.java (DependencyRule 含 suggestion)

```

package com.archchecker.domain.rule;

import com.archchecker.domain.codebase.File;
import com.archchecker.domain.compliance.Violation;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;

import java.io.IOException;
import java.nio.file.Files;

```

```

import java.nio.file.Path;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class DependencyRuleTest {

    @Test
    void disallowedImportProducesViolation(@TempDir Path tmp) throws IOException {
        Path src = writeJava(tmp, "C",
            "package com.foo.domain;\nimport com.foo.infra.X;\n\nclass C {}\n");
        DependencyRule rule = new DependencyRule("R-DEP-01",
            "domain must not depend on infra",
            "com.foo.domain", "com.foo.infra", false);

        List<Violation> result = rule.validate(List.of(new File(src, "com.foo.domain")));

        assertEquals(1, result.size());
        assertTrue(result.get(0).getMessage().contains("must not depend on"));
    }

    @Test
    void allowedImportsAreIgnored(@TempDir Path tmp) throws IOException {
        Path src = writeJava(tmp, "C",
            "package com.foo.app;\nimport com.foo.domain.Y;\n\nclass C {}\n");
        DependencyRule rule = new DependencyRule("R-DEP-02",
            "app must depend on domain",
            "com.foo.app", "com.foo.domain", true);

        List<Violation> result = rule.validate(List.of(new File(src, "com.foo.app")));

        assertTrue(result.isEmpty(), "expected no violation but got " + result);
    }

    @Test
    void filesOutsideSourcePackageAreSkipped(@TempDir Path tmp) throws IOException {
        Path src = writeJava(tmp, "C",
            "package com.bar;\nimport com.foo.infra.X;\n\nclass C {}\n");
        DependencyRule rule = new DependencyRule("R-DEP-01",
            "domain must not depend on infra",
            "com.foo.domain", "com.foo.infra", false);

        List<Violation> result = rule.validate(List.of(new File(src, "com.bar")));

        assertTrue(result.isEmpty());
    }

    private static Path writeJava(Path dir, String name, String body) throws IOException {
        Path p = dir.resolve(name + ".java");
        Files.writeString(p, body);
        return p;
    }
}

```

7. Revision Based on TA Comments, Midterm Review, and Findings

本次 Iteration II 新增 UC-02（Define Style Profile）與 UC-03（Check with Interactive Fix Suggestions）之設計與實作，以下說明兩個 use case 的主要設計決策。

UC-02 Design（Define Style Profile）：

- 新增 profile init 與 profile validate 兩個子命令，分別對應 initProfile(outputPath) 與

validateProfile(yamlPath) 兩個系統事件。ProfileInitCommand 擔任 Controller，ProfileTemplateService 作為 Information Expert 負責產生 YAML 範本；ProfileValidateCommand 擔任 Controller，ProfileValidateService + YamlProfileLoader 共同實現驗證邏輯。

- YAML 格式沿用 SnakeYAML 與既有 YamlProfileLoader 整合，維持低耦合（Low Coupling）：ProfileValidateService 依賴 ProfileLoader 介面而非具體實作，便於替換。

UC-03 Design (Check with Interactive Fix Suggestions)：

- 新增 fix 子命令，由 FixCommand 作為 Controller 接收 fix(sourcePath, profilePath) 系統事件；FixCommand 先呼叫 ComplianceCheckService 取得 ViolationReport，再逐一顯示違規並讀取 Developer 決策（action: y | n）。

- SuppressionService 作為 Pure Fabrication，封裝 suppress 流程（讀取現有清單、建立 Suppression 記錄、寫回 YAML）；FixCommand 於 action == "y" 時委派 SuppressionService.suppress(violation, profile, suppressFile)，無需了解 SuppressionStore 的讀寫細節，達成 Indirection 與 Low Coupling。

- Domain Model 僅新增 Violation.suggestion [0..1] 屬性，遵循 Information Expert 原則——修正建議最直接屬於「單筆違規」，而非規則或其他物件；suggestion 為純文字值，不應升格為獨立概念類別。

8. Measurement

Student ID	Name	HW	Date	Duration
113598009	李俊威	HW#1	115/03/03 15:00 ~ 18:00	3H
		HW#2	115/03/12 15:00 ~ 21:00	6H
		HW#3	115/04/01 14:00 ~ 20:00	6H
		HW#4	115/04/17 14:00 ~ 18:00	4H
			115/04/22 13:00 ~ 17:00	4H
		HW#5	115/04/29 19:00 ~ 23:30	4.5H
			115/05/05 15:00 ~ 23:00	8H
		HW#6	115/05/26 20:00 ~ 23:30	3.5H
		HW#7	115/06/17 13:00 ~ 20:00	7H